LAB 1499

Hands-On with Docling for IBM watsonx

Convert, extract, and build your own document-powered app

Michele Dolfi dol@zurich.ibm.com
Senior Technical Staff Member, IBM Software

Cesar Berrospi Ramis ceb@zurich.ibm.com
Senior Researcher, IBM Software

Panos Vagenas pva@zurich.ibm.com
Advisory Engineer, IBM Software

Peter Staar taa@zurich.ibm.com
Principal RSM, Manager, IBM Software



Notices and disclaimers

© 2026 International Business Machines Corporation. No part of this document may be reproduced or transmitted in any form without written permission from IBM.

U.S. Government Users Restricted Rights — use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM.

This document is current as of the initial date of publication and may be changed by IBM at any time. Not all offerings are available in every country in which IBM operates.

Information in these presentations (including information relating to products that have not yet been announced by IBM) has been reviewed for accuracy as of the date of initial publication and could include unintentional technical or typographical errors. IBM shall have no responsibility to update this information.

This document is distributed “as is” without any warranty, either express or implied. In no event, shall IBM be liable for any damage arising from the use of this information, including but not limited to, loss of data, business interruption, loss of profit or loss of opportunity. IBM products and services are warranted per the terms and conditions of the agreements under which they are provided. The performance data and client examples cited are presented for illustrative purposes only. Actual performance results may vary depending on specific configurations and operating conditions.

IBM products are manufactured from new parts or new and used parts. In some cases, a product may not be new and may have been previously installed. Regardless, our warranty terms apply.

Any statements regarding IBM's future direction, intent or product plans are subject to change or withdrawal without notice.

Performance data contained herein was generally obtained in a controlled, isolated environments. Customer examples are presented as illustrations of how those customers have used IBM products and the results they may have achieved. Actual performance, cost, savings or other results in other operating environments may vary.

References in this document to IBM products, programs, or services does not imply that IBM intends to make such products, programs or services available in all countries in which IBM operates or does business.

Workshops, sessions and associated materials may have been prepared by independent session speakers, and do not necessarily reflect the views of IBM. All materials and discussions are provided for informational purposes only, and are neither intended to, nor shall constitute legal or other guidance or advice to any individual participant or their specific situation.

It is the customer's responsibility to ensure its own compliance with legal requirements and to obtain advice of competent legal counsel as to the identification and interpretation of any relevant laws and regulatory requirements that may affect the customer's business and any actions the customer may need to take to comply with such laws. IBM does not provide legal advice or represent or warrant that its services or products will ensure that the customer follows any law.

Notices and disclaimers (continued)

Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products. IBM does not warrant the quality of any third-party products, or the ability of any such third-party products to interoperate with IBM's products. IBM expressly disclaims all warranties, expressed or implied, including but not limited to, the implied warranties of merchantability and fitness for a purpose.

The provision of the information contained herein is not intended to, and does not, grant any right or license under any IBM patents, copyrights, trademarks or other intellectual property right.

IBM, the IBM logo, and ibm.com are trademarks of International Business Machines Corporation, registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the Web at “Copyright and trademark information” at: www.ibm.com/legal/copytrade.shtml.

Python is a trademark of the Python Software Foundation. Jupyter is a trademark of NumFOCUS, Inc. All company names, product names and scenarios used in this lab's sample documents are fictitious.

Table of contents

1 Introduction 6
1.1 A two-minute introduction to Docling 6
1.2 A two-minute introduction to DocLang 7
1.3 The managed service, and what it means for you 8
1.4 What you will build 8
1.5 The shape of the lab 9
1.6 Prerequisites 10
1.7 Conventions 10
2 Getting started 11
2.1 Connect to your lab VM 11
2.2 Set up the lab 12
2.3 What the bootstrap left you 13
3 Part 1 — Get ready with Docling for IBM watsonx 15
3.1 Start your trial 15
3.2 Convert a document, and look at what comes out 17
3.3 Export the document to DocLang 18
3.4 Inspect the DocLang file 20
3.5 Get your service URL and API key 22
3.6 Put them in your .env file 23
4 Part 2 — Choose your track 25
4.1 Where both tracks are going: the form 25
5 Track A — Document intelligence with IBM Bob 28
5.1 Check the connection 28
5.2 Log in to Bob 29
5.3 The docling-lab skill 31
5.4 Convert, explore, and extract 31
5.5 Fill the form from all three documents 33
5.6 Write the summary and the filled form 34
5.7 If time permits — the form as DocLang 35
6 Track B — Document intelligence with Python 37

6.1 Open the notebook in Bob 37
6.2 Run the setup 38
6.3 Work through it 38
6.4 Query DocLang from the command line 39
6.5 The filled form 40
6.6 If time permits — fill the form with Docling Agent 42
7 Compare the two routes 43
7.1 What you just did 43
8 Where to take it next 45
9 Did not finish the lab? 45
10 Getting help 45
10.1 Ask the lab instructor 45
10.2 Common problems 46
10.3 Useful commands 47
11 Reference material 47

1 Introduction

You are a compliance analyst at Vantara Bank, a mid-sized bank operating across several regulatory jurisdictions. Every quarter a bundle lands on your desk — a risk disclosure as a PDF, an internal audit summary as a Word document, a set of regulatory guidelines as an EPUB — and from it you must fill in one standardised Compliance Assessment Form. The numbers are in the PDF, the findings are in the DOCX, the obligations are in the EPUB, and none of the three is in the shape the form wants.

Today you automate that: from three raw documents to a filled-in, audit-ready assessment, without standing up a single GPU.

Two things make it possible, and the lab is really about the first one:

- **A document has structure, and losing it is the whole problem.** Extracting text from the PDF gives you a wall of characters in which a table is indistinguishable from a paragraph. Reconstructing the document gives you a table you can read a cell out of. Everything downstream — extraction, retrieval, the form — is bounded by which of those two you started from.
- **You do not have to run any of it yourself.** Docling for IBM watsonx does the reconstruction as a managed service. You send a document and get a structure back.

You will do that first in a browser with no code at all, and then from a tool of your choosing — an AI assistant or a Python notebook — which takes you all the way to the filled form. At the end you compare the two routes.

1.1 A two-minute introduction to Docling

Docling is an open-source document-processing toolkit, started at IBM Research in Zurich and now hosted by the LF AI & Data Foundation. **Docling for IBM watsonx** is the supported, hosted version of it — the one you will use today.

What it does is narrower and deeper than “extract the text”:

- **It reconstructs the document.** Page layout, reading order, headings, lists, tables, formulas, code blocks and figures — recovered as structure, not as a flat string. A two-column paper comes out in reading order; a table comes out as a table.
- **It reads more than PDF.** DOCX, PPTX, XLSX, HTML, Markdown, EPUB, images and much more. Scanned pages go through OCR. All of them come out as the same thing, a `DoclingDocument`, which is why today's three very different files can be asked the same question.

- **It chunks.** The chunker is token-aware and structure-aware: it splits on the document's own boundaries, keeps a chunk's headings attached to it, and sizes chunks to the token budget of the model that will consume them.

> **NOTE**
> Everything you do after Part 1 is bounded by what conversion produced. A field that was never recognised as a table cell cannot be extracted from one, and a passage that was separated from its heading cannot be cited convincingly no matter how good the model reading it is. This is why the first thing you will do is look at a conversion result closely, even though nothing later strictly requires it.

1.2 A two-minute introduction to DocLang

Once a document has been reconstructed, it has to be written down somewhere. **DocLang** (`.dclx`) is an open, XML-based ISO standard for exactly that — what JSON is for data and HTML is for pages, DocLang is for documents that have been understood.

Markdown is the obvious alternative and it is what most pipelines use. The difference matters as soon as you want to *extract* rather than *read*:

	Markdown	DocLang
A heading	a line starting with `##`	a node, with its level and its children
A table	pipes and dashes, if it survived at all	rows, cells and column headers, each addressable
A figure caption	a paragraph near an image	a caption **belonging to** its figure
“the third table in section 2”	a string search and some hope	an XPath: `/heading[2]/table[3]`
Page furniture	mixed into the text	a layer you can switch off

The format, its specification and its tooling are described at doclang.ai. The **DocLang Viewer**, at doclang.ai/viewer, opens a `.dclx` file in your browser and shows the original page, the DocLang markup and a reading view side by side; the file never leaves your machine.

You will export a document to DocLang in Part 1, open it in that viewer, and switch the furniture layer off to see that headers and footers were never part of the content in the first place. In Track B you will then query one from the command line with `dclq`, and the notebook can write the filled form itself as a DocLang file.

1.3 The managed service, and what it means for you

Docling can run on your own machine, and for a lot of work it should. Today it runs as a service, for three reasons that are worth naming because they are the reasons a team would make the same choice:

- **No models to download and no GPU to find.** Layout analysis, table structure recovery and OCR are real models. The service has them warm.
- **The call shape does not change.** `DoclingServiceClient` takes the same arguments as the local `DocumentConverter`. Code written against one moves to the other by swapping the constructor.
- **It scales past one laptop.** `convert_all()` sends a batch and the service works through it concurrently.

What you give up is that the document leaves your machine, which is a real consideration and the reason the local path exists. For a lab with three fictional documents it is not one.

The language models are hosted too. Track A uses the model behind IBM Bob; Track B calls **watsonx.ai** for embeddings and generation. Nothing on the VM runs a model.

1.4 What you will build

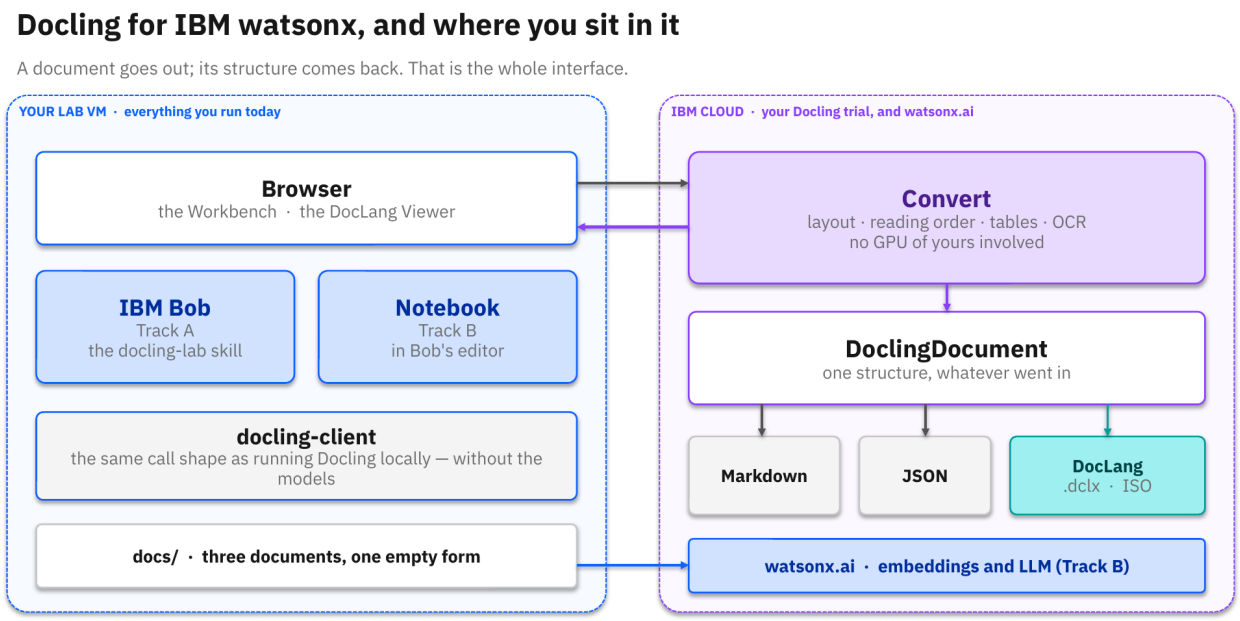


Figure 1. Everything on the left runs on the lab VM in front of you; everything on the right is your own Docling trial on IBM Cloud. The only thing crossing the middle is a document going one way and its structure coming back.

Note what is **not** on the diagram: any model running on the VM. The VM holds a browser, an IDE, a terminal and a thin client library. All the document understanding happens on the right.

1.5 The shape of the lab

One common start, two routes to the form

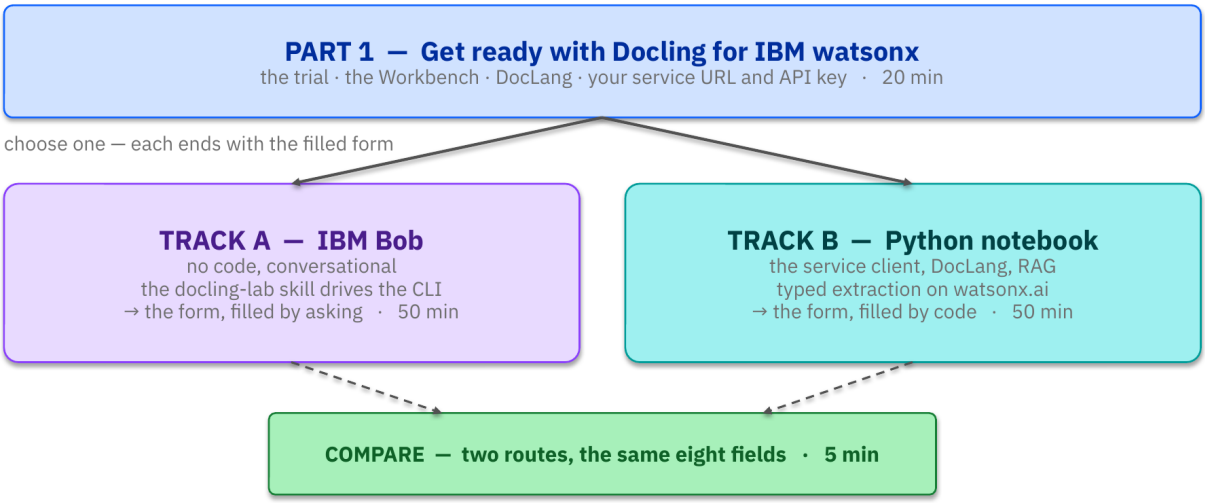


Figure 2. Part 1 is common. Then you pick a track, and the track takes you all the way to the filled compliance form. The last five minutes compare the two routes.

Part	What you do	Time
Part 1	Get ready with Docling for IBM watsonx — the trial, the Workbench, DocLang, your credentials	20 min
Part 2	**Track A** with IBM Bob, *or* **Track B** with the Python notebook — each ends with the filled form	50 min
Compare	Two routes to the same eight fields	5 min
Total		**75 min**

The session is 90 minutes and the activities are sized for 75. The rest is setup, questions, and the extension steps at the end of each track for anyone who moves quickly.

> **NOTE**
>
> **Every part works on its own.** If Part 1 runs long — provisioning is somebody else's cloud and occasionally slow — ask the instructor for a service URL and key and start Part 2 with those.

1.6 Prerequisites

	How you get it
A **lab VM**	provided for you — a Linux desktop you reach through the browser. **Everything in this lab runs on it**; nothing needs to be installed on your own laptop.
An **IBMid**	your own, free. The same IBMid signs you in everywhere today: the lab, the Docling trial and IBM Bob.
A **Docling for IBM watsonx trial**	you create it in §3.1, on that IBMid. It stays yours after the lab.
IBM Bob	already installed on the VM. Track A talks to its assistant; Track B uses it as an IDE to run the notebook.
watsonx.ai *(Track B)*	an API key and a project ID shared by the class. The instructors put them on your VM before the lab and the bootstrap copies them into place — nothing to type. watsonx.ai serves the embedding model and the language model the notebook uses.

1.7 Conventions

- Grey boxes are commands you type in a terminal **on the lab VM**. Lines starting with `#` are comments, not commands.
- **Bold** is something in the interface: a button, a menu item, a field label.
- Text in `this typeface` is something you type, or something the system shows you literally.
- Coloured boxes — NOTE, TIP, WATCH OUT, YOU NOW HAVE — are worth reading before you continue. The last of them closes each part and tells you what you should be holding at that point.

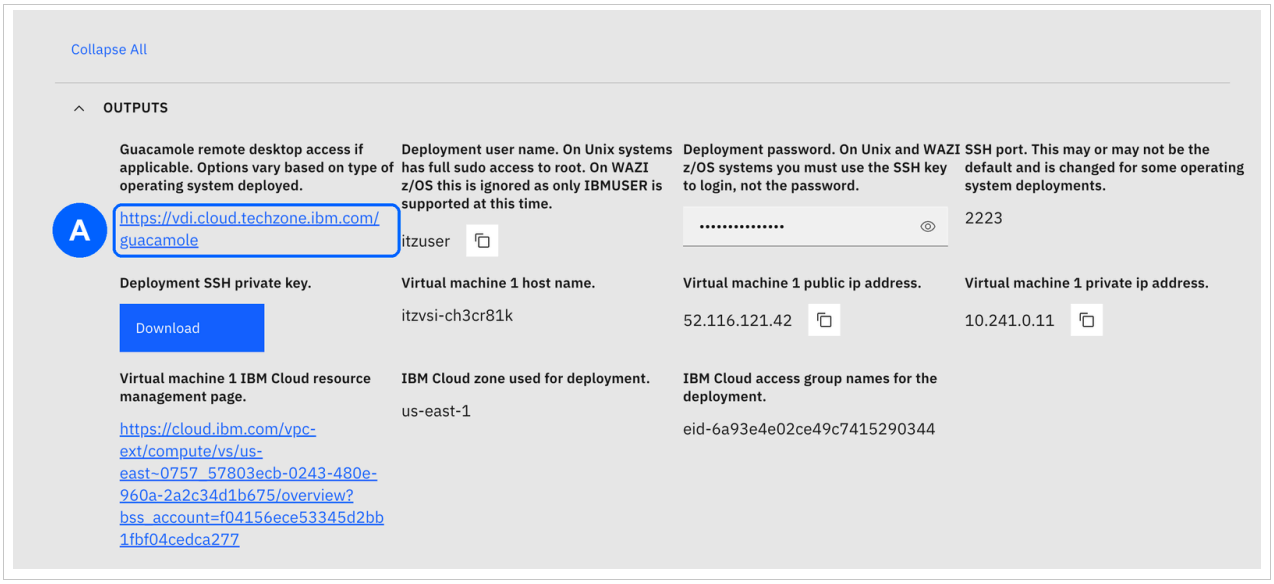
2 Getting started

Five minutes of setup that everything after it depends on. Do all of it before Part 1.

2.1 Connect to your lab VM

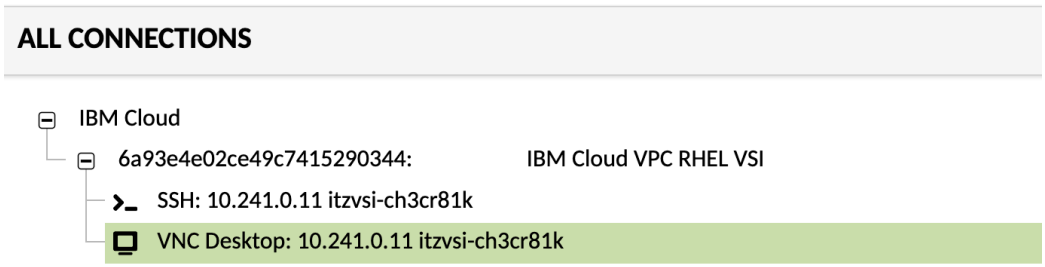
Everything in this lab runs on a Red Hat Enterprise Linux virtual machine with a desktop, which you reach through your own browser. It has Firefox, a terminal and the IBM Bob IDE already installed. Nothing runs on your laptop.

1. Open your lab details page — the one you land on after registering — and expand its **Outputs**.
2. Open the **remote desktop access** link (A).

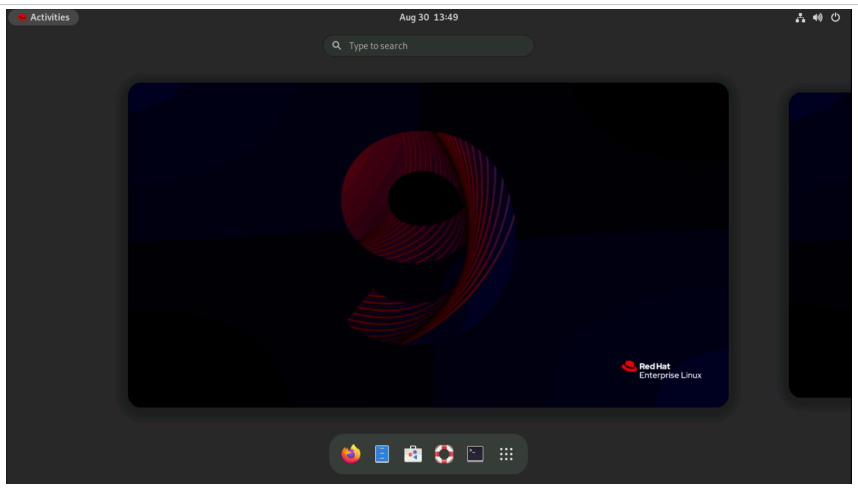


Screen 1. The environment's **Outputs** panel. The remote-desktop link (A) is the only thing on it this lab needs.

3. Under **All connections**, choose the **VNC Desktop** entry — not the SSH one. SSH gives you a shell without a browser, and most of Part 1 happens in a browser.
4. You should get a desktop with Firefox, a terminal and IBM Bob. Leave the browser open — you will use it for the trial, the Workbench and the DocLang Viewer.



Screen 2. Two connections are offered for the same machine. Take the highlighted one.



Screen 3. The desktop you land on. Firefox and the terminal are in the dock at the bottom, and IBM Bob beside them.

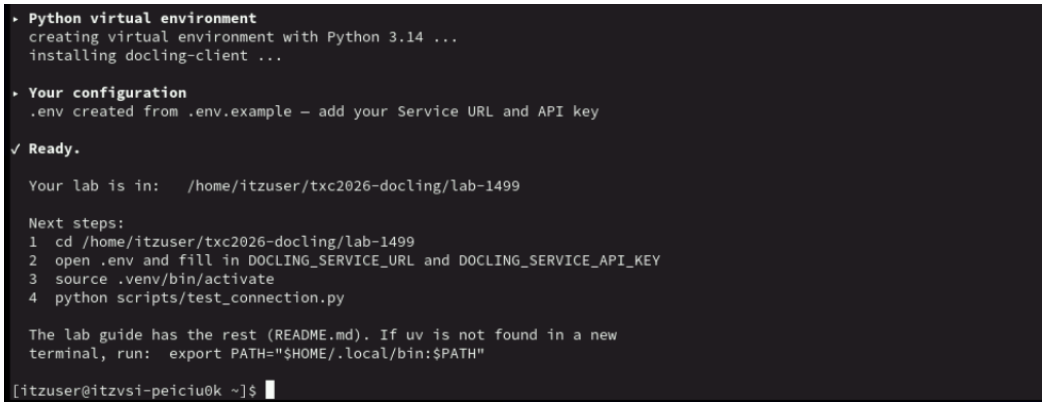
> **TIP**
>
> The remote desktop is a browser tab like any other, so it competes with your laptop for keyboard shortcuts. If a copy or paste does not seem to arrive, click once inside the desktop first to give it focus.

2.2 Set up the lab

Open a terminal on the VM and run one command:

```
curl -L ibm.biz/txc26-1499-bootstrap | bash -
```

It clones the lab repository, installs the system packages the lab needs, installs `uv`, creates the Python environment and puts the Docling client in it. It takes a minute or two, and it is safe to run again at any point — a second run updates the checkout and leaves your own `.env` alone.



Screen 4. The end of a successful run. It has cloned the lab, built the environment and written your `.env` — so §3.6 only has to fill it in.

When it finishes, go to the lab directory:

```
cd ~/txc2026-docling/lab-1499
```

> **NOTE**
> If a command is *not found* immediately afterwards, you are in the terminal that existed before the tooling was installed. Run `export PATH="$HOME/.local/bin:$PATH"` in it, or just open a new one.

2.3 What the bootstrap left you

Four things in that directory are worth knowing about now:

	What it is
`docs/`	the three documents you will process, `compliance_assessment_form.json` — the empty form both tracks fill in — and `docling_lab.ipynb`, the Track B notebook
`.env.example`	the template for your credentials. The bootstrap copied it to `.env`, which you fill in at §3.6; that one file is what both tracks read
`.bob/skills/docling-lab/`	the skill that teaches IBM Bob how to use Docling. Track A uses it; Bob finds it on its own
`scripts/test_connection.py`	a one-command check that your credentials work, before you depend on them

YOU NOW HAVE

A desktop on the lab VM, the lab cloned into `~/txc2026-docling/lab-1499`, and a Python environment with the Docling client in it.

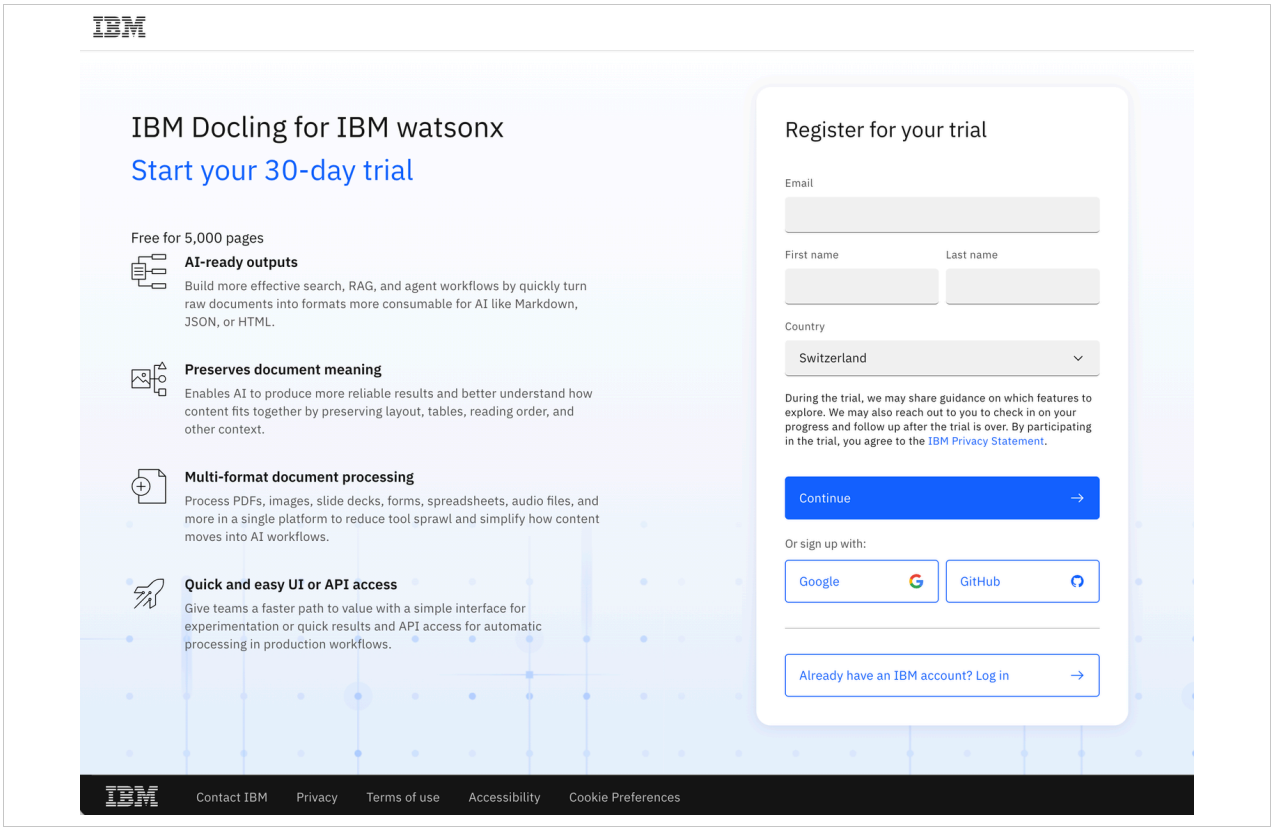
3 Part 1 — Get ready with Docling for IBM watsonx

Twenty minutes, everyone. You start your own trial, convert a document in the Workbench and look hard at what comes out, export it to DocLang and inspect it, and finish holding the two values the rest of the lab needs: a service URL and an API key.

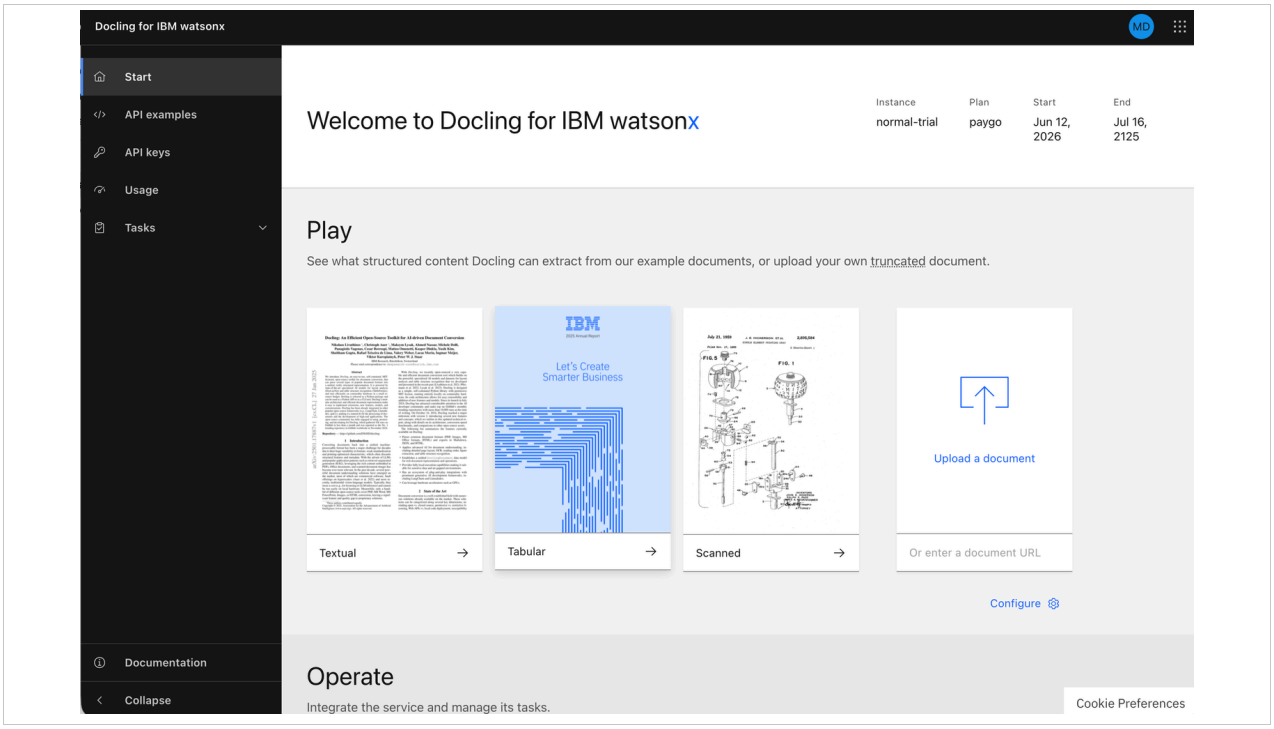
3.1 Start your trial

1. In the browser **on the lab VM**, go to ibm.biz/docling-for-watsonx-trial-txc26.
2. Sign in with your IBMid, or create one — it is free.
3. Follow the prompts to start the trial. Provisioning takes about a minute; wait until it says *Your trial is ready!*
4. Click **Access your trial now**. You land on the Docling for IBM watsonx dashboard.

> **TIP**
>
> Already signed up for a trial earlier and not sure how to get back to your Docling instance? Go to ibm.biz/mydocling.



Screen 5. The trial registration page. Registering with an existing IBMid is the **Already have an IBM account? Log in** link at the bottom of the panel.

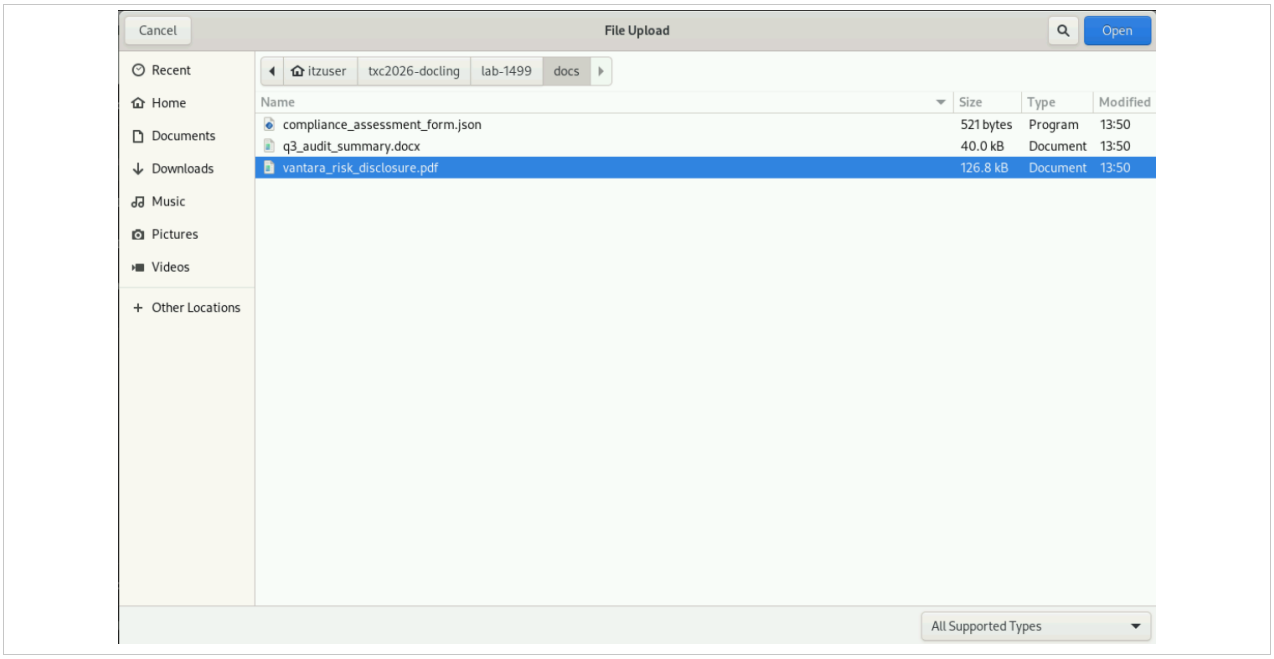


Screen 6. The dashboard once the instance is active. **Play** converts a document straight away; **API examples** and **API keys** in the sidebar are where §3.5 goes.

3.2 Convert a document, and look at what comes out

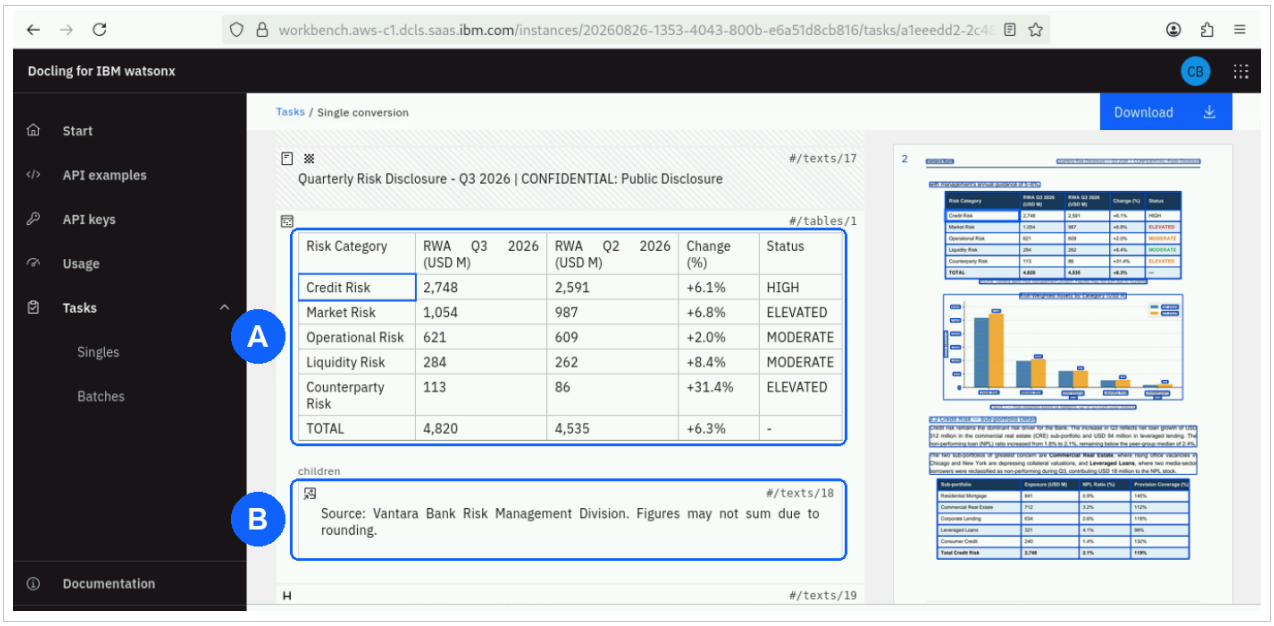
This step produces nothing that later steps need. Do it anyway: this is the only point in the lab where you see conversion directly rather than through a client library, and everything downstream is shaped by it.

1. On the dashboard, under **Play**, choose to upload a document. In the file chooser click **Home**, then work down through `txc2026-docling` → `lab-1499` → `docs`, select `vantara_risk_disclosure.pdf` and click **Open**.



Screen 7. The file chooser, at the end of that path. The lab's three documents are all in `docs/`; this is the one to take now.

2. When conversion finishes you are taken to **Tasks**. The left column is the document as a tree of structural nodes; the right column renders the source page with what was recognised drawn over it.



Screen 8. The Vantara risk disclosure, converted. (A) is the risk-category table, recovered as a **table** — separate rows, columns and header cells, addressable as `#/tables/1`. (B) is the source note underneath it, kept attached to the table it belongs to instead of drifting into the page's running text. Two of the eight fields you fill at the end of your track are read straight out of (A).

3. Click a **table** node and check that its cells and column headers came out separately, rather than as one run of text.
4. Find a two-column page, if the document has one, and check the **reading order** — the tree should not interleave the columns.
5. Switch the output format between Markdown and JSON. The JSON is the structured document; the Markdown is one rendering of it.
6. Try it again with `docs/q3_audit_summary.docx`, or any URL you like. A DOCX and a PDF produce the same kind of tree — that is the point.

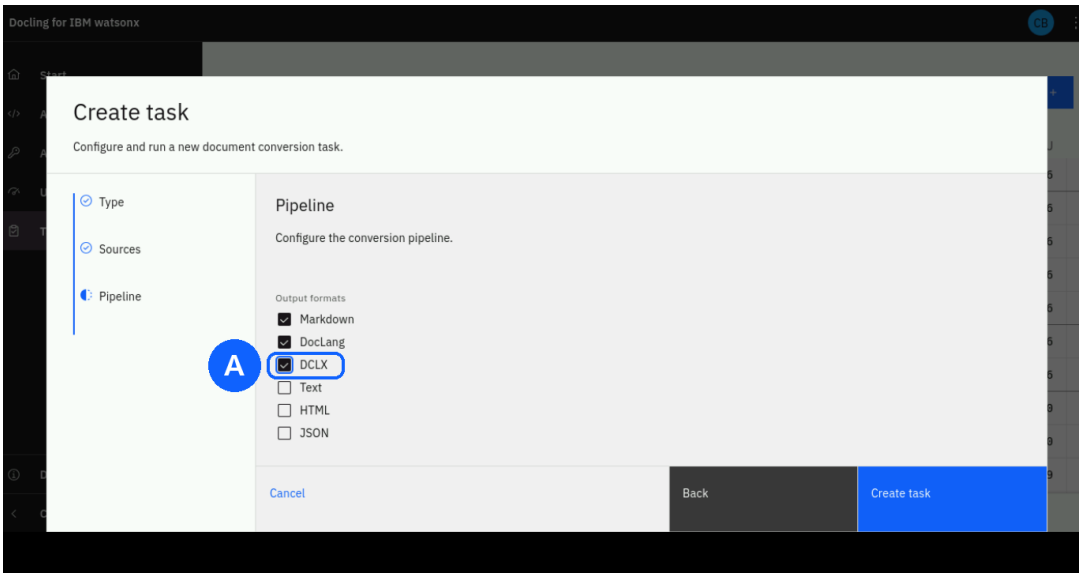
> **NOTE**
>
> What you are looking at is what the rest of the lab operates on. The eight form fields at the end of both tracks are read out of this structure; if a number lives in a table cell that was recovered as a cell, asking for it is a lookup. If the table collapsed into prose, it is a guess.

3.3 Export the document to DocLang

Now produce the same document as a `.dclx` file, which §3.4 opens in a viewer.

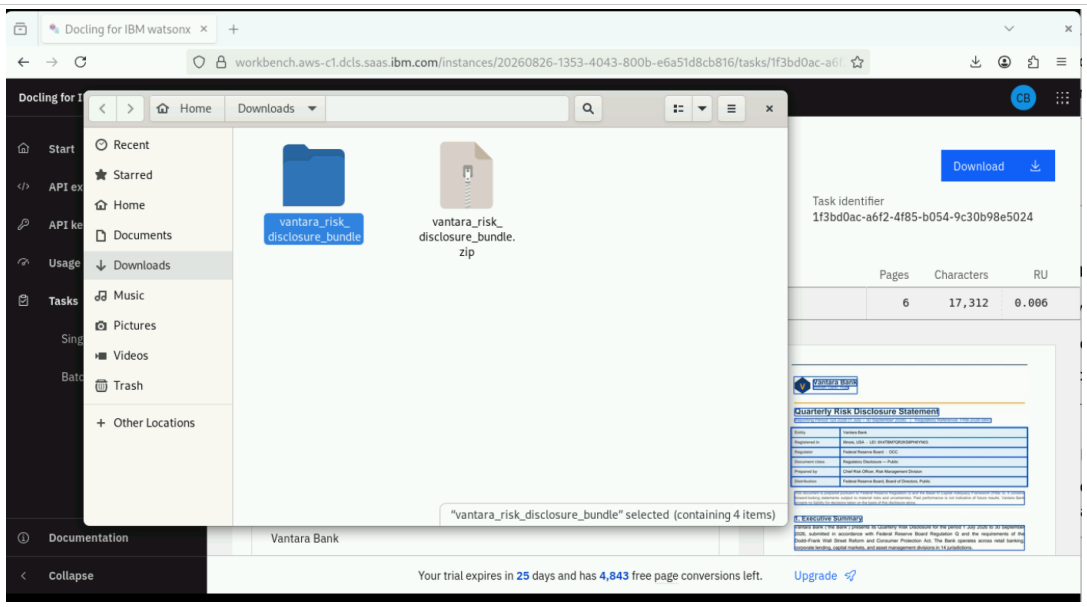
1. Go to **Tasks → Create task → Single → Next**.
2. Select `vantara_risk_disclosure.pdf` again.

3. Under **Output formats**, check **DCLX**.
4. Run the task. When it finishes, click **Actions**, then **Download** to save the bundle.



Screen 9. The last step of the wizard. **DCLX** (A) is the one this section needs; the others are free, so leave them on if you want to compare the formats afterwards.

5. The download is a zip — `vantara_risk_disclosure_bundle.zip`. Open it from your **Downloads** folder and extract it; the `.dclx` you need is inside.

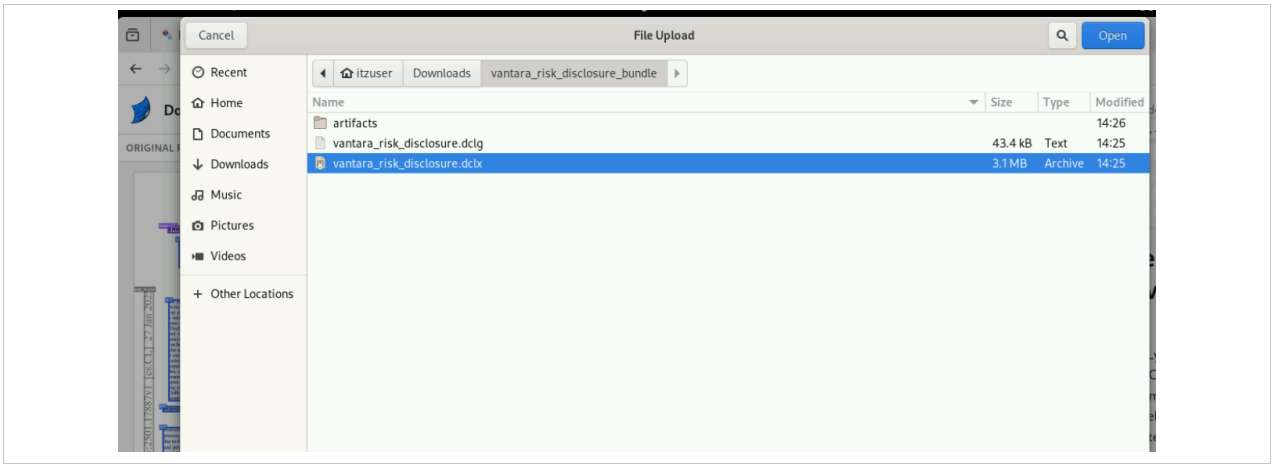


Screen 10. `Downloads` after extracting: the bundle folder beside the zip it came from. The folder holds one file per output format you ticked.

3.4 Inspect the DocLang file

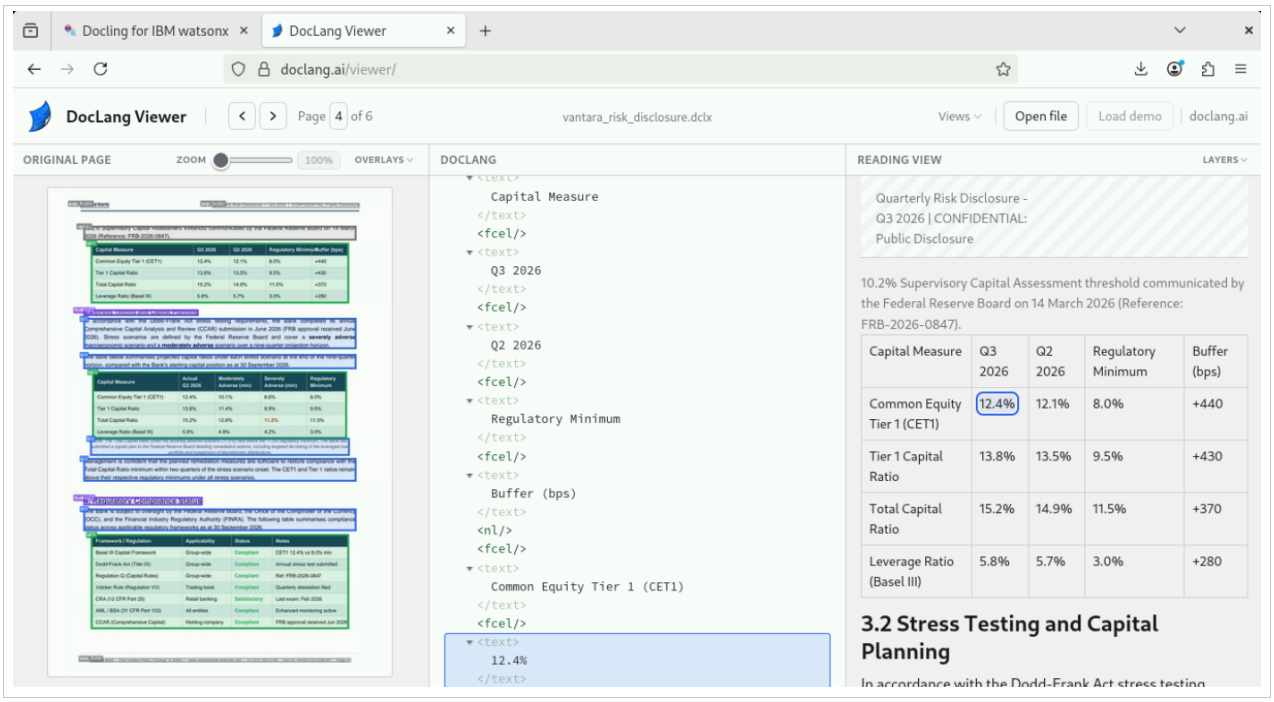
The viewer is a plain web page — the file never leaves your browser.

1. Open doclang.ai/viewer and use **Open file**. Take the `.dclx` from the **folder you just extracted**, not the zip itself.

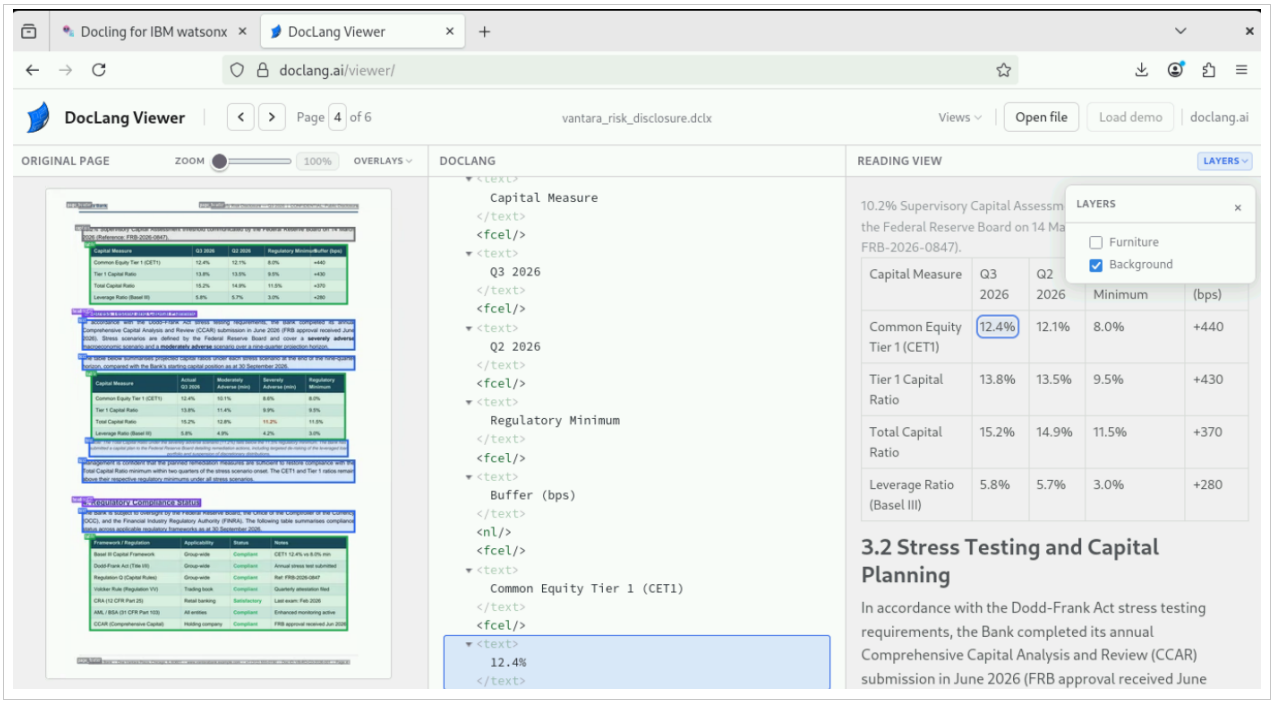


Screen 11. Inside the extracted bundle. `vantara_risk_disclosure.dclx` is the one the viewer wants.

2. You get three panels: **Original page** on the left with the recognised regions drawn over it, **DocLang** in the middle — the XML itself — and **Reading view** on the right.
3. Click a table cell in the reading view. It lights up in all three panels at once: the cell, its element in the XML, and its box on the original page. That correspondence is the whole point of the format.
4. Open **Layers** at the top right of the reading view and uncheck **Furniture**. The running header at the top of the page disappears — it was never part of the content.



Screen 12. The three panels, with one figure — `12.4%` — selected in all of them at once. The hatched block at the top of the reading view is page furniture.



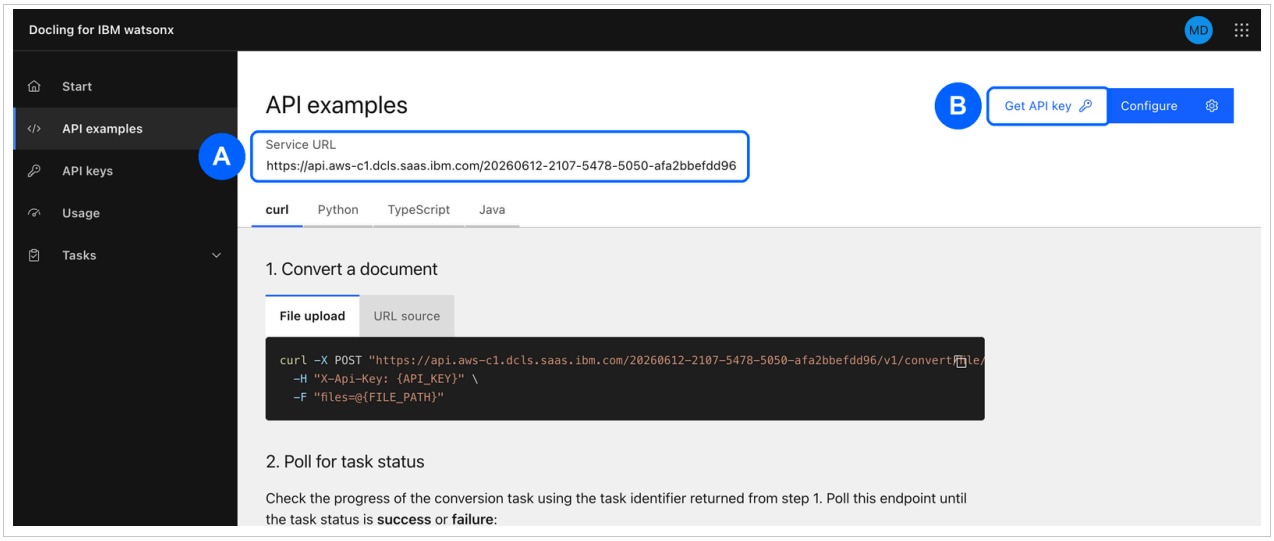
Screen 13. The same document with **Furniture** unchecked. The hatched header is gone and the content starts where the content actually starts.

> **NOTE**
> That furniture layer is a small thing that matters a lot in practice. A page footer repeated on all forty pages of a report is forty chunks of noise in a retrieval index, and separating it at conversion time is much easier than filtering it out later.

3.5 Get your service URL and API key

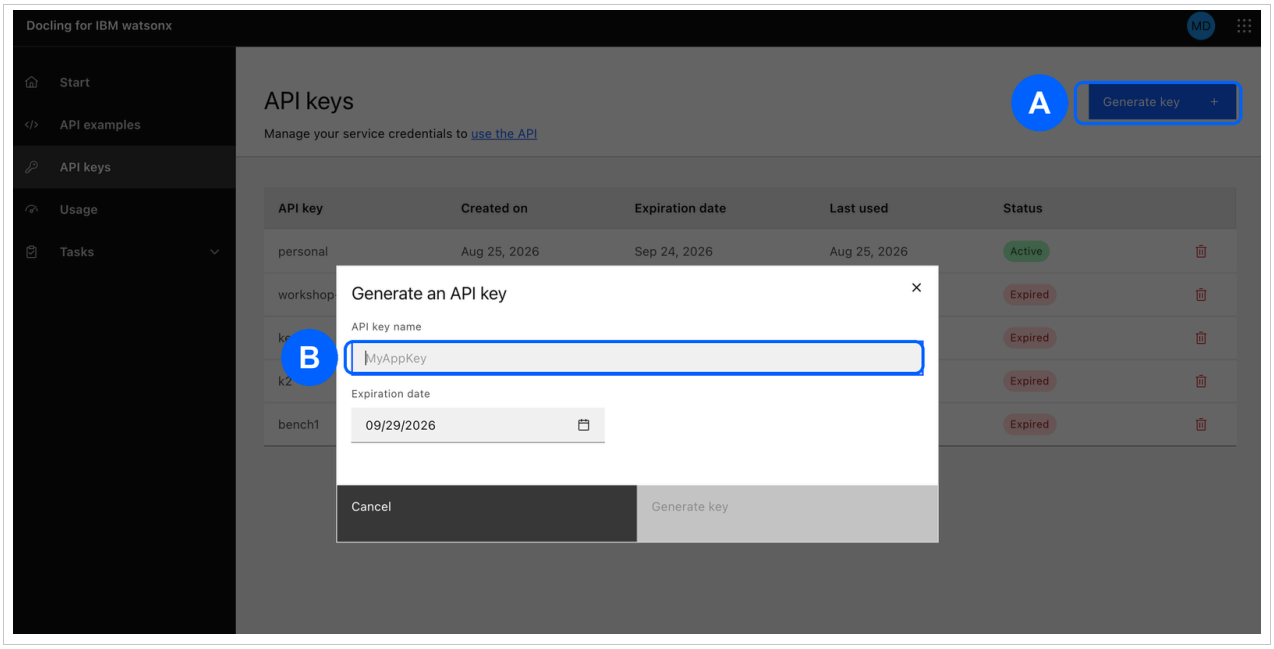
These two values are what connects the tools in Part 2 to **your** instance.

1. Open **API examples** in the sidebar. The **service URL** (A) is at the top — copy the whole of it, including the path after the host name: the part after the host is what identifies your instance.
2. Follow **Get API key** (B) — or **API keys** in the sidebar.



Screen 14. The service URL (A), and the way to the keys (B). The `curl` example below them is the same call the client library makes for you in Part 2.

3. Click **Generate an API key**, give it a name — `txc26-lab` will do — and confirm.
4. Copy the key.



Screen 15. Generate key (A) opens the dialog; the key needs a name (B) and nothing else.

WATCH OUT

The API key is shown **once**. If you close the dialog without copying it, generate another — there is no way to read an existing key back.

The address in your browser's URL bar is the **dashboard**, not the service URL. They are different host names for the same instance, and only the service URL works in Part 2.

3.6 Put them in your .env file

Both tracks read the same file, so this is done once and neither track asks again. The bootstrap in §2.2 already created it from `.env.example`; all that is left is to fill it in.

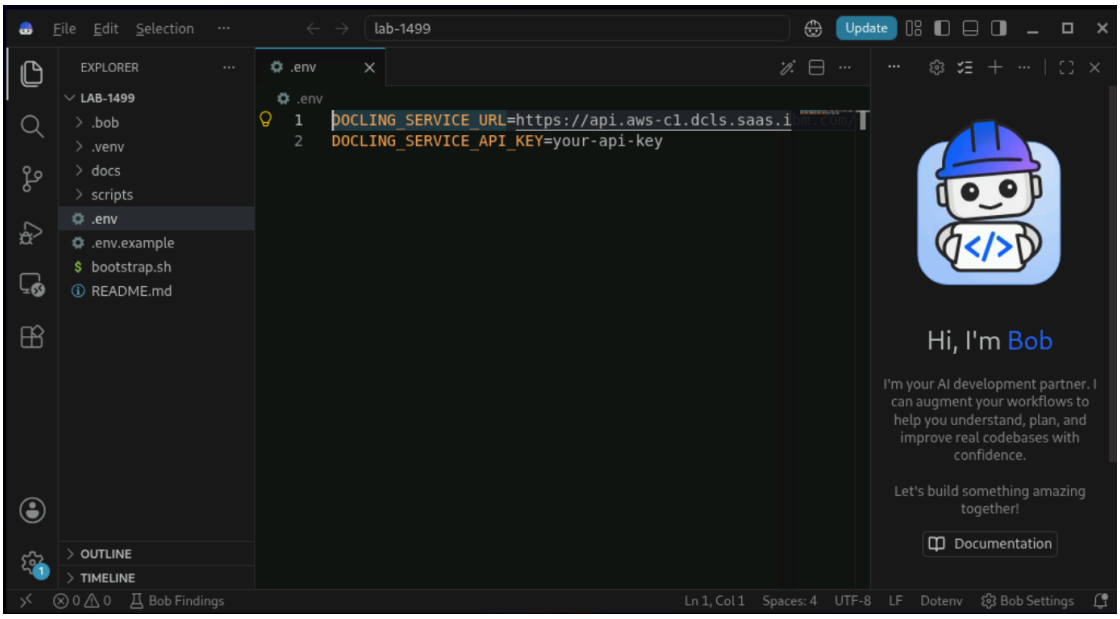
Open `.env` — in IBM Bob's file explorer, or in any editor on the VM — and replace the two placeholder values:

```
DOCLING_SERVICE_URL=https://<the service URL you copied>
DOCLING_SERVICE_API_KEY=<the key you copied>
```

Further down the same file are three `WX_` lines: the **watsonx.ai** credentials Track B uses. They are shared by the whole class, so you do not type them — the instructors put them on your VM in `~/lab-1499-watsonx.txt` before the lab, and the bootstrap copied them into `.env`. Leave them as they are. Track A does not use them.

TIP

If the `WX_` lines still show `your-…` placeholders, the bootstrap ran before the file arrived. Run the bootstrap again (§2.2) — it keeps everything you typed — or open `lab-1499-watsonx.txt` from your home folder and copy its lines into `.env`. The notebook also reads that file on its own when `.env` has no value.



Screen 16. `.env` open in IBM Bob. It sits beside `.env.example` in the lab directory; edit the copy without the suffix, and save.

YOU NOW HAVE

Your own Docling instance, a document you have looked at closely enough to know what conversion produces, and a `.env` file with a working service URL and API key.

4 Part 2 — Choose your track

Fifty minutes, and this is where the lab splits for good. Both tracks use the same service, the same three documents and the same credentials, and both end with the same thing: the compliance assessment form, filled in. What differs is what you drive it from — a conversation, or code you can read. Section 7 compares the two.

	Track A — IBM Bob	Track B — Python notebook
Best for	anyone; no coding required	developers, and anyone who wants the API
You drive it from	a conversation with Bob's assistant	notebook cells, in Bob's editor
What it teaches	how an assistant with a document skill behaves, and where its limits are	the service client, `dclq`, chunking, RAG and typed extraction on watsonx.ai
You finish with	the form as JSON, a summary, and the form as a Markdown table	the form as a typed object, an HTML report, and the form as a DocLang file
Needs	IBM Bob, already on the VM	the shared watsonx.ai credentials, already on your VM (§3.6)
Read	Section 5	Section 6

> **TIP**
> If you cannot decide: take Track A if you want to see what this looks like in the hands of someone who will never write Python, and Track B if you want to know what happens underneath. Neither is the advanced one.

4.1 Where both tracks are going: the form

The form needs eight fields and no single document holds them all, which is the part that makes this a pipeline rather than a query.

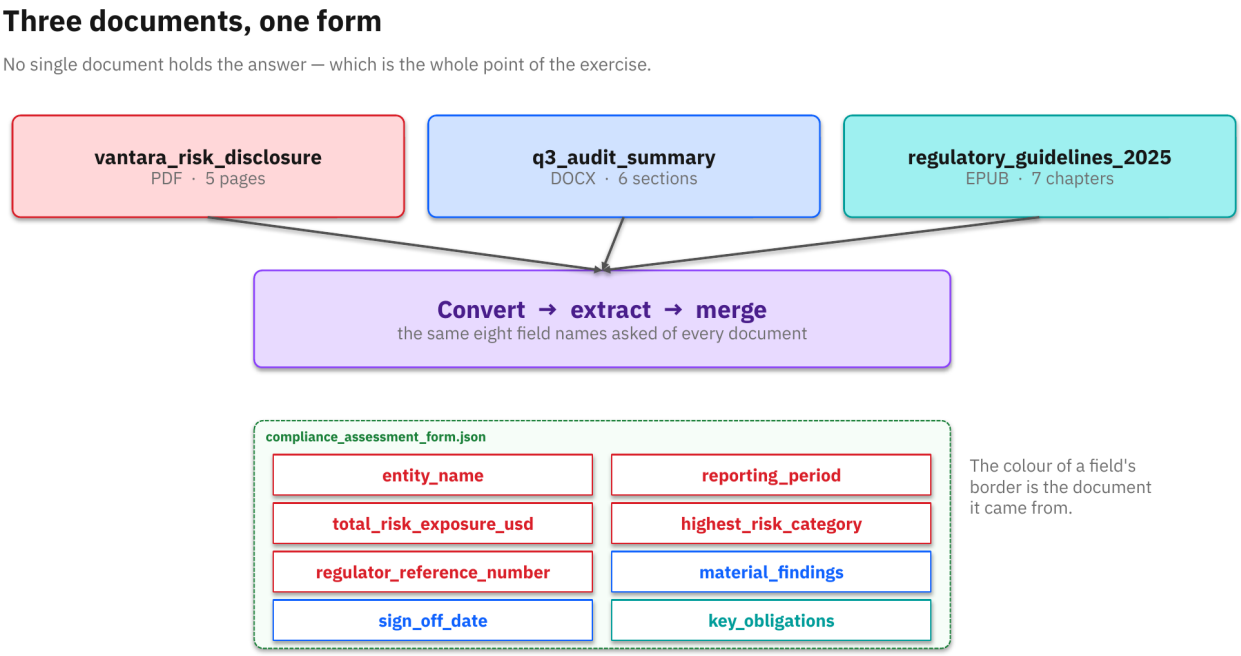


Figure 3. The eight fields of the compliance assessment form, coloured by the document they come from. Five are in the risk disclosure, two in the audit summary and one in the regulatory guidelines — so all three have to be processed.

The empty form is `docs/compliance_assessment_form.json`:

```
{
  "entity_name": null,
  "reporting_period": null,
  "total_risk_exposure_usd": null,
  "highest_risk_category": null,
  "regulator_reference_number": null,
  "key_obligations": [],
  "material_findings": [],
  "sign_off_date": null
}
```

Field	Lives in
`entity_name`, `reporting_period`, `total_risk_exposure_usd`, `highest_risk_category`, `regulator_reference_number`	`vantara_risk_disclosure.pdf`
`material_findings` (list), `sign_off_date`	`q3_audit_summary.docx`
`key_obligations` (list)	`regulatory_guidelines_2026.epub`

NOTE

The spread is deliberate. Two of the fields — the risk exposure figure and the regulator's reference number — appear in **two** documents, which is what lets you check one against the other. A pipeline that reads only the PDF gets an answer; a pipeline that reads all three can tell you whether the answer is consistent.

Whichever track you take, go to **Section 7** when your fifty minutes are up, even if you have not finished: the comparison takes five minutes.

5 Track A — Document intelligence with IBM Bob

TRACK A

IBM Bob — no code required

You are in the no-code track. Everything here happens in a conversation with IBM Bob and in one terminal, and it ends with the filled form. If you meant to take the Python track, that is **Section 6**.

IBM Bob is an AI assistant with an IDE around it. On its own it knows nothing about Docling; the lab repository carries a skill that teaches it, and Bob picks that up when it opens the workspace. What you will do is check the connection, confirm the skill is there, ask about one document, and then ask for the whole form.

5.1 Check the connection

Before asking an assistant to use a service, confirm the service answers. Go back to the terminal from §2.2 and activate the environment the bootstrap created:

```
cd ~/txc2026-docling/lab-1499
source .venv/bin/activate
```

Then run the check. It converts `docs/regulatory_guidelines_2026.epub` and prints the first lines of the Markdown that comes back — which confirms the URL is reachable, the key is valid and conversion works, all three at once:

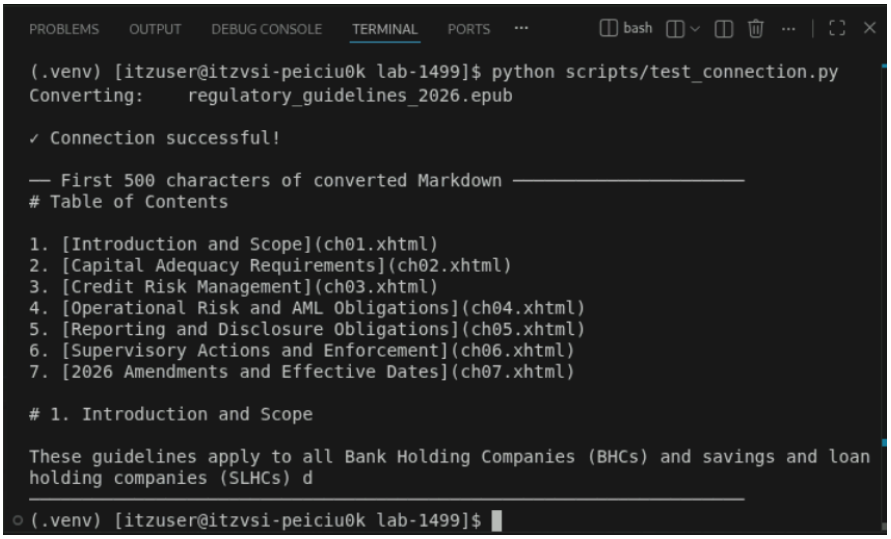
```
python scripts/test_connection.py
```

A successful run looks like this:

```
Connecting to: https://<your-service-url>
Converting:    regulatory_guidelines_2026.epub

√ Connection successful!

── First 500 characters of converted Markdown ──────────────────
## Q3 2026 Internal Audit Summary
...
```



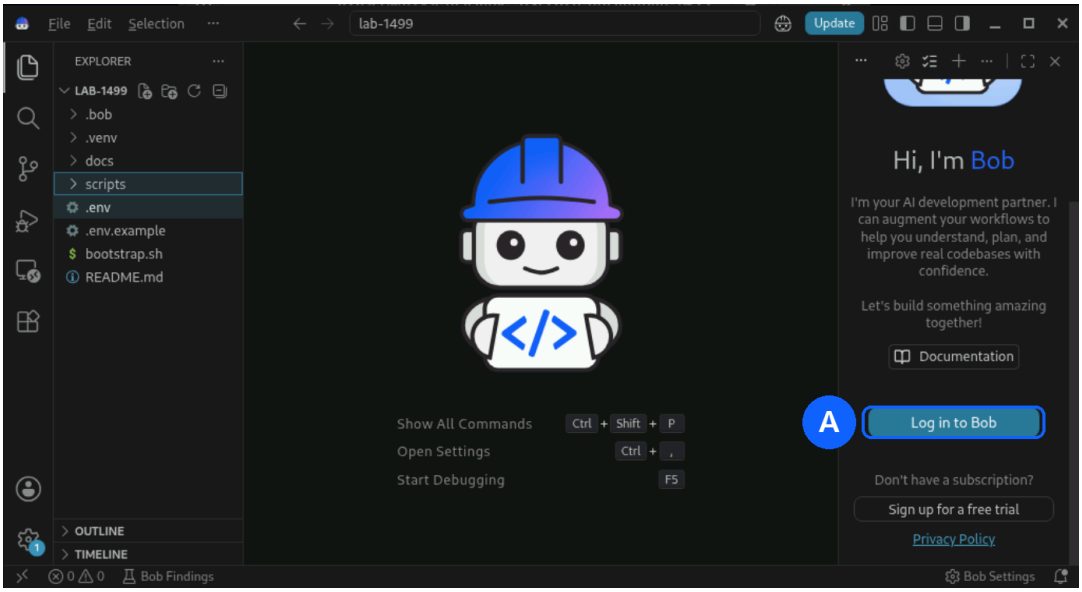
Screen 17. A successful check. The Markdown underneath the tick is the EPUB, converted by your own instance — which is three things confirmed in one command.

If you see	It means	Go to
`X Missing credentials`	`.env` is absent, or a variable name is misspelt	§3.6
`401` or `403`	the API key is wrong, or was regenerated after you copied it	§3.5
a connection error	the service URL is wrong — most often the dashboard URL was copied instead	§3.5

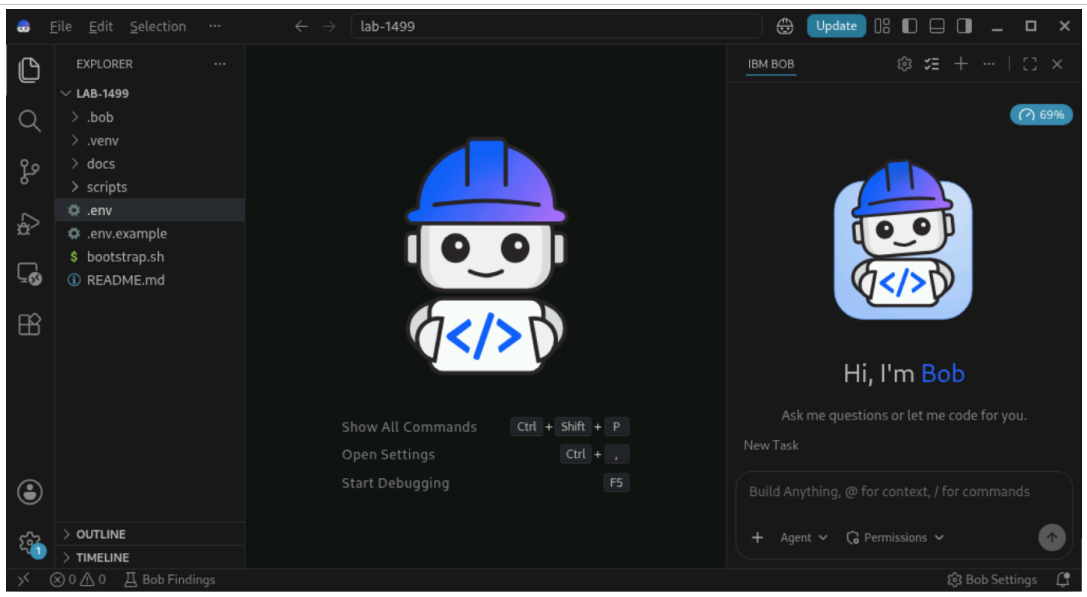
5.2 Log in to Bob

Bob is installed on the VM but not yet signed in, and it signs in as **you** — with the same IBMid you used for your Docling trial.

1. In the panel on the right, click **Log in to Bob**.
2. You get the message *The extension 'IBM Bob' wants to sign in using Bob.* Click **Allow**, and then **Open**.
3. A new Firefox tab opens. Sign in to ibm.com with your IBMid credentials.
4. When it finishes the panel changes: the **Log in to Bob** button is gone and a message box appears in its place, under *Ask me questions or let me code for you*. That box is where every prompt in this track goes.



Screen 18. Before signing in. Bob greets you either way, so the greeting is not the thing to look for — **Log in to Bob** (A) is, and so is the offer of a free trial underneath it.



Screen 19. After. The button has been replaced by a message box, and the panel is now headed **IBM Bob**. This is the state the rest of Track A assumes.

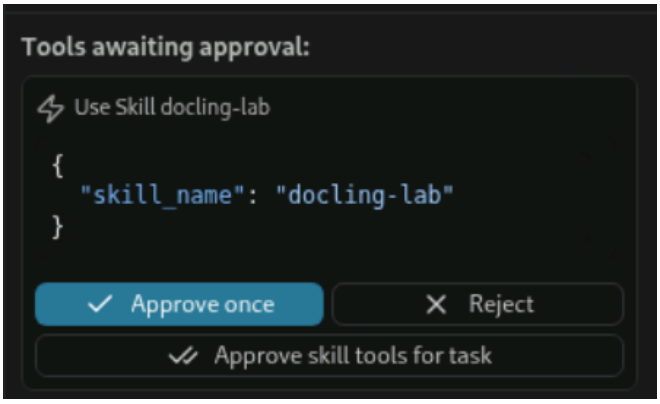
> **TIP**
>
> One IBMid covers everything today — the lab, the Docling trial and Bob. If a sign-in page asks, it is always the same account.

5.3 The docling-lab skill

The lab repository ships a skill at `.bob/skills/docling-lab/SKILL.md`. It teaches Bob to convert documents with the `docling convert-remote` command line — one binary against your service, with nothing to keep running in the background.

```
ls .bob/skills/docling-lab/SKILL.md
```

Bob scans `.bob/skills/` when it opens the workspace, so the skill is available from your first message. The first time it reaches for it you will see **Tools awaiting approval: Use Skill docling-lab** — choose **Approve skill tools for task**.



Screen 20. The one prompt you have to answer. Approving for the task means Bob will not ask again for the rest of this conversation.

> **NOTE**
> A skill is a plain Markdown file. Open `SKILL.md` and read it — it is a short description of a tool and when to use it, and that is the entire mechanism. Nothing about it is specific to Bob or to Docling, which is why the same file works for any assistant that reads skills.

5.4 Convert, explore, and extract

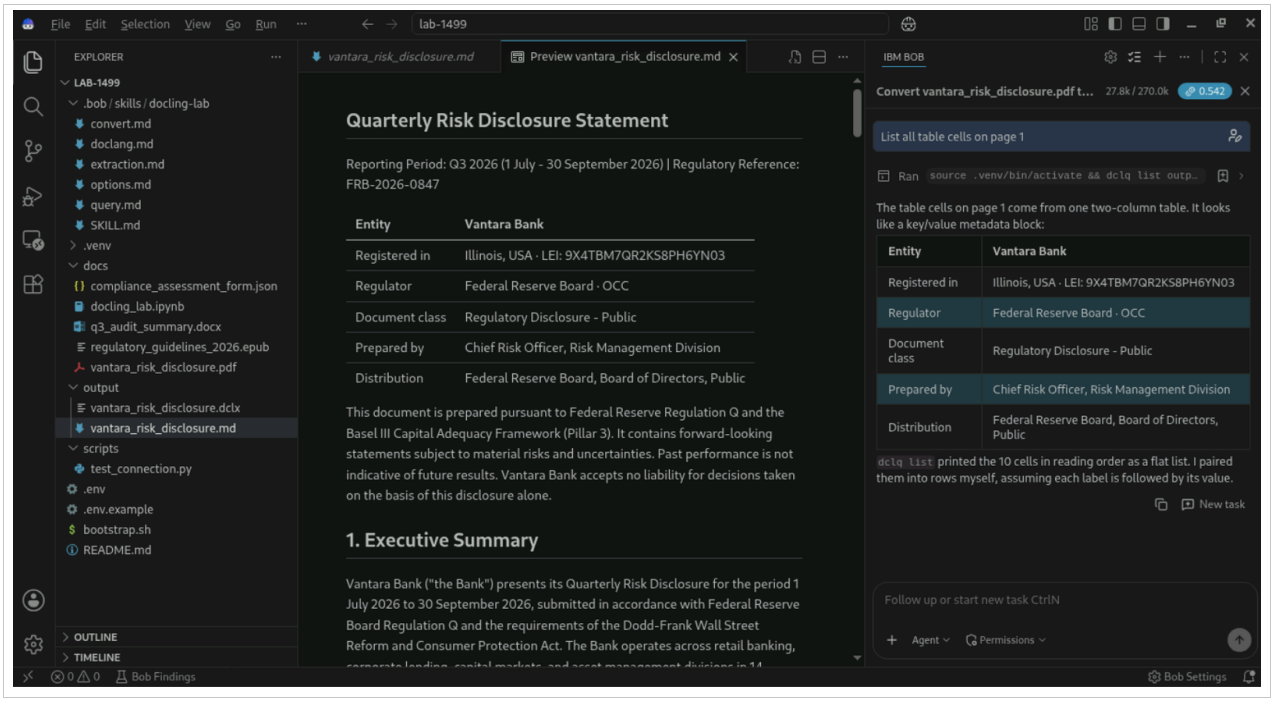
Now use it. Type these one at a time in the Bob chat, and read what comes back rather than only that it came back:

```
Convert vantara_risk_disclosure.pdf to Markdown and DocLang.
```

```
Show me the heading outline.
```

```
Show me the Executive Summary section.
```

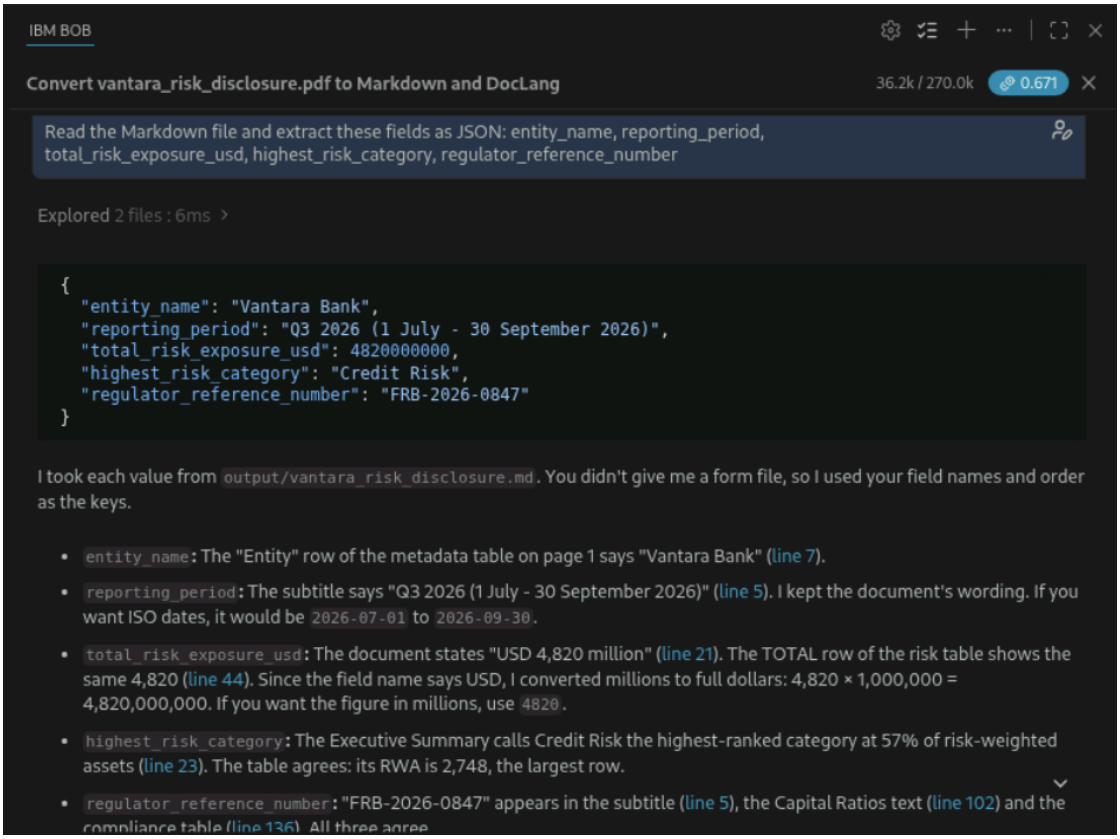
```
List all table cells on page 1
```



Screen 21. The conversation on the right, the result in the middle. The conversion left `vantara_risk_disclosure.md` and `vantara_risk_disclosure.dclx` in `output/`, and the Markdown is open in the centre panel. For the last prompt Bob *ran a command* — `dclq list` on the DocLang file — and got back the cells of the metadata table on page 1, the same ones you see in the preview.

The last ones are the interesting ones. Bob did not read the PDF — it read the structure Docling produced from it, which is why the answer follows the document's own sections. Now ask for something in a fixed shape:

```
Read the Markdown file and extract these fields as JSON: entity_name,
reporting_period, total_risk_exposure_usd, highest_risk_category,
regulator_reference_number.
```



Screen 22. The same document, asked for five named fields — and Bob says which line of the Markdown each one came from. `total_risk_exposure_usd` comes back as `4820000000`: the `4,820` in the TOTAL row of the table you looked at in §3.2, in millions. It is right because that row survived conversion as a row.

> **TIP**
>
> Check the extracted numbers against the document. This is a language model reading a structure, not a parser reading a schema, and knowing where it is reliable is worth more than a clean-looking result. The five fields above all come from the same document on purpose; the next step is where it gets harder.

5.5 Fill the form from all three documents

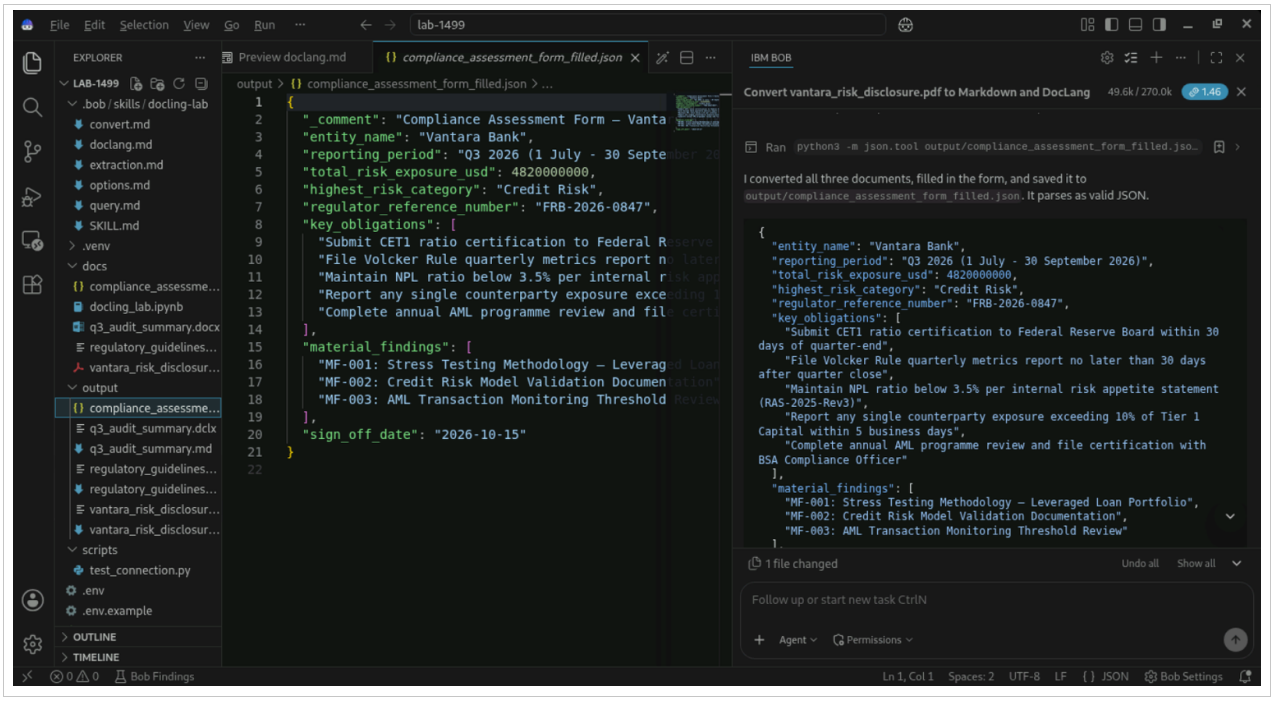
The form in §4.1 needs all three documents. Convert the other two first, and look at the audit summary before asking it anything — it is a DOCX with a different shape from the PDF:

```
Convert q3_audit_summary.docx and regulatory_guidelines_2026.epub to DocLang.
```

```
Show me the outline of the audit summary.
```

Then ask for all fields from the form at once:

```
Convert the documents q3_audit_summary.docx, regulatory_guidelines_2026.epub, and
vantara_risk_disclosure.pdf, extract the fields from the form
compliance_assessment_form.json, and save the results in a JSON file.
```



Screen 23. The filled form. Bob saved it as `output/compliance_assessment_form_filled.json`, open in the editor, and checked that it parses as valid JSON — the eight fields, with the five scalars from the PDF, the findings and sign-off date from the DOCX, and the obligations from the EPUB.

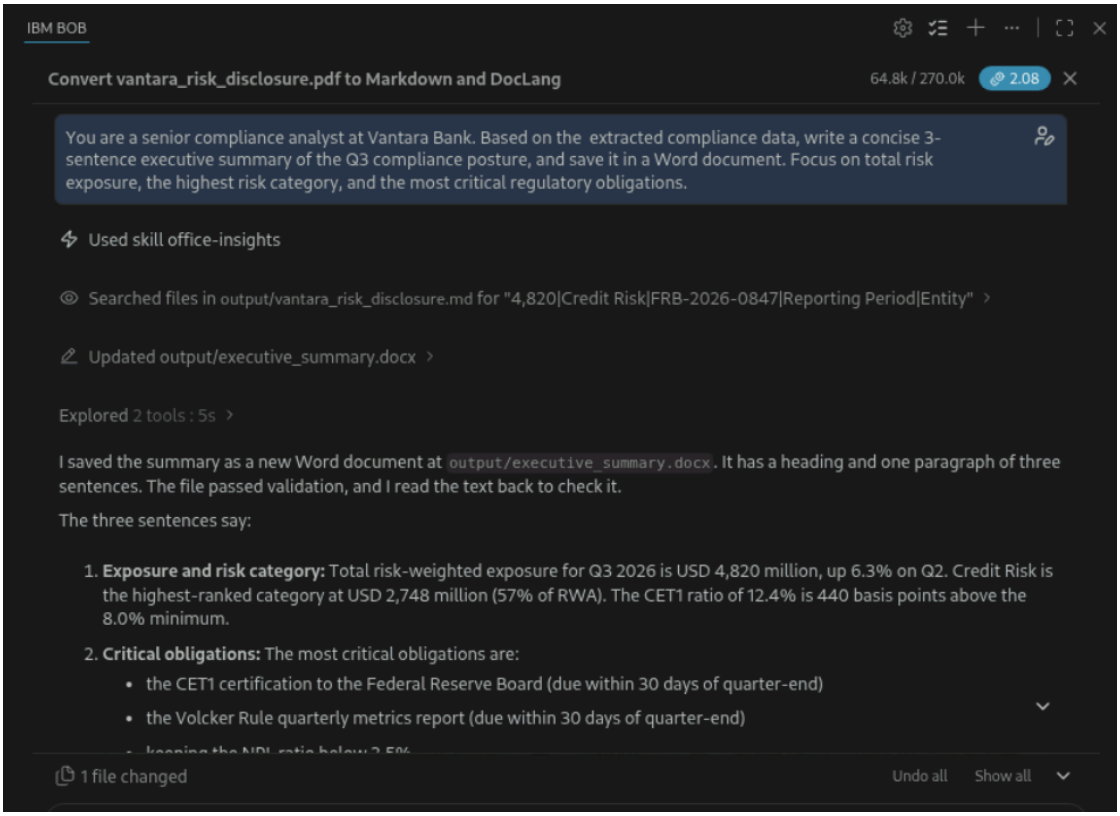
For the last prompt Bob reads the Markdown files it converted and saves the extracted values to a JSON file — the same `compliance_assessment_form.json` template the notebook fills in Track B. Read the chat to see where each value came from.

> **TIP**
>
> Look at what happened to the two fields that appear twice. If both documents agreed, combining them is trivial. If they did not, Bob made a choice — and in a real compliance pipeline that choice is the thing an auditor would ask about. Ask Bob which document each value came from.

5.6 Write the summary and the filled form

The last step turns the structured result back into something a person reads:

```
You are a senior compliance analyst at Vantara Bank. Based on the extracted
compliance data, write a concise 3-sentence executive summary of the Q3 compliance
posture, and save it in a Word document. Focus on total risk exposure, the highest
risk category, and the most critical regulatory obligations.
```



Screen 24. The summary, saved as `output/executive_summary.docx`. The trace shows how: **Used skill office-insights**, a search of the converted Markdown for the figures it quotes, and then the Word file written and read back to check it.

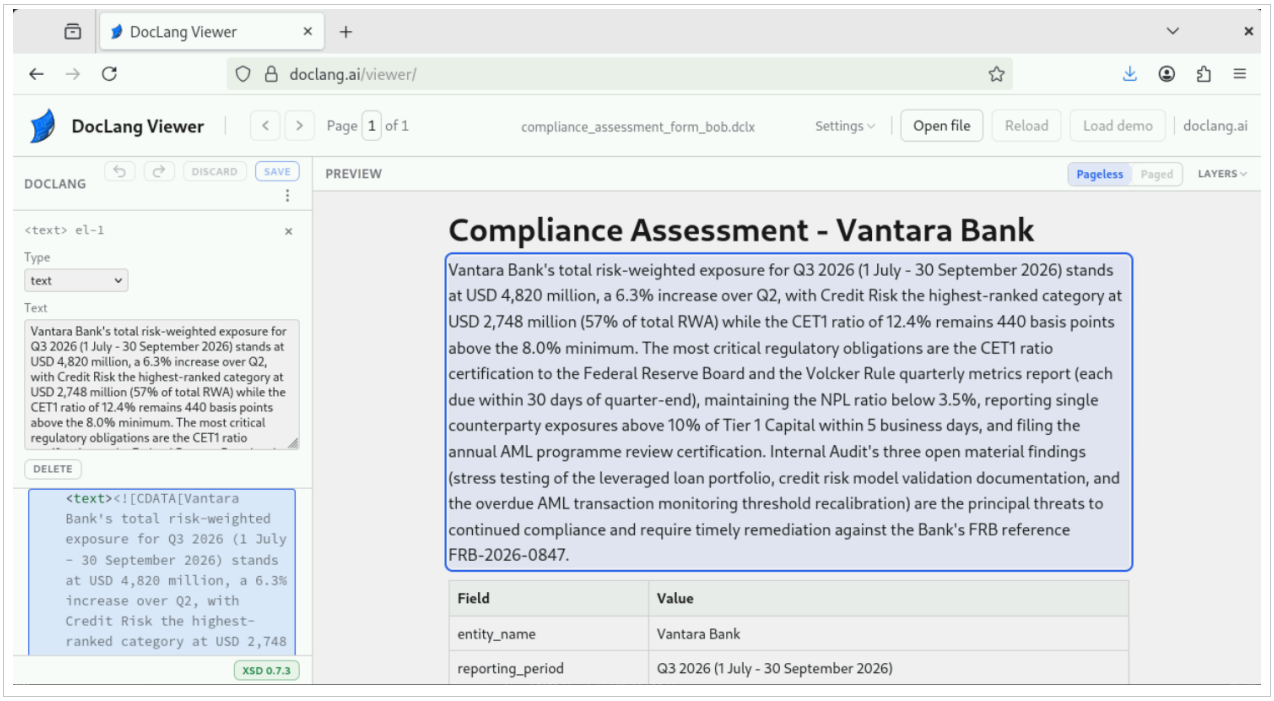
For writing the executive summary in Word, Bob uses one of its many built-in skills and tools (**office-insights**).

5.7 If time permits — the form as DocLang

Now build the compliance report as a proper DocLang document — title, summary, and form table — and open it in the viewer:

```
Using the executive summary and the filled form from the previous steps, build a
DoclingDocument named "compliance_assessment_form" with:
- a title "Compliance Assessment Form — Vantara Bank"
- the executive summary as a text block
- the eight form fields as a two-column table (Field | Value)
Save it as output/compliance_assessment_form_bob.dclx.
```

Drag the file onto doclang.ai/viewer. The form you asked for in a conversation is a regular DocLang document, with the same kind of title, table and cells as a converted PDF.



Screen 25. Bob's report in the DocLang Viewer: the title, the executive summary as one text element — selected, so its DocLang source shows on the left — and the form underneath as a Field | Value table.

YOU NOW HAVE

The compliance assessment form filled from three documents, a written summary, and the form as a document — without writing any code yourself. Go to **Section 7**.

6 Track B — Document intelligence with Python

TRACK B

Python notebook — the API underneath

You are in the developer track. Here **IBM Bob is your IDE**: you use its editor, its file Explorer and its terminal to run the lab's notebook, as you would in any other IDE. You do not need Bob's AI assistant in this track. If you meant to take the no-code track, that is **Section 5**.

The notebook goes all the way from the three documents to the filled form, and you can read every step: the client library, the DocLang command line, the chunker, a RAG pipeline, typed extraction with a Pydantic model, and an agent that edits a document. Nothing runs a model on the VM — Docling converts on your trial instance, and watsonx.ai serves the embedding model and the language model.

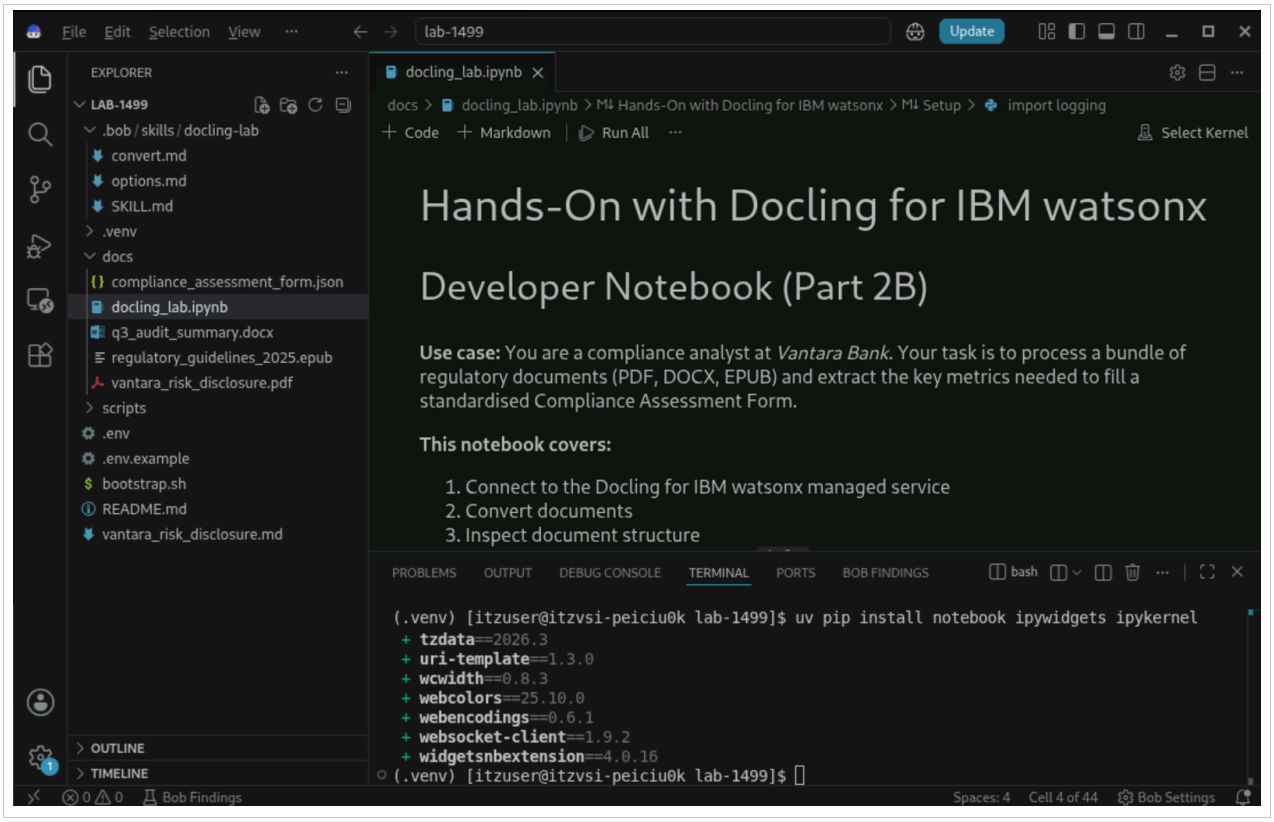
6.1 Open the notebook in Bob

Bob is an IDE, so it runs the notebook itself — there is no separate Jupyter to start and no second browser tab to keep track of.

1. Open a terminal inside Bob: **Terminal → New Terminal** from the menu bar. It opens in the lab directory.
2. Activate the environment the bootstrap made, and add the package that lets an IDE run notebook cells in it:

```
source .venv/bin/activate
uv pip install ipykernel
```

3. In the **Explorer** on the left, open `docs` and click `docling_lab.ipynb`. It opens in the middle panel.
4. Click **Select Kernel** at the top right and choose the lab's `.venv`. A notebook with no kernel runs nothing.



Screen 26. The three things you need, in one window: the terminal at the bottom, the Explorer on the left with the notebook selected under `docs/`, and the notebook itself in the middle.

6.2 Run the setup

1. Run the first cell under **Setup**. It installs the packages the notebook uses — all of them thin clients, with no machine-learning framework and no model weights.
2. Run the second one. It finds the lab folder, loads your `.env` and checks the values are there.

> **NOTE**
>
> The notebook reads the same `.env` you filled in at §3.6, so there is nothing to paste into a cell. For the watsonx.ai values it also reads `~/lab-1499-watsonx.txt`, the file the instructors put on your VM. If the setup cell still names a missing variable, see §10.2.

6.3 Work through it

Run the notebook sections in order. Each one is short, and the text cells say what to look at.

Section	What you do	Worth stopping on
1	Connect to the managed service with `DoclingServiceClient`	the constructor is the only line that differs from running Docling locally
2	Convert the PDF, the DOCX and the EPUB with `convert_all()`	three formats, one result type
3	Inspect the structure — sections, tables, figures	this is the tree you saw in the Workbench, as objects
4	Export all three documents to DocLang and query them with `dclq`	`outline` prints the XPath of every heading
5	Chunk the documents with `HybridChunker`	chunks follow the headings and fit the token budget of the embedding model
6	A RAG pipeline: watsonx.ai embeddings, Milvus, and a watsonx.ai model to answer	every answer lists the document, page and headings it came from
7	Typed extraction: the LLM fills a Pydantic `ComplianceForm` from all three documents	the schema is code, and the eight fields come back typed
8	Write the executive summary and the HTML report	the report is the filled form, in `output/`
9	*(optional)* Fill the form with `DoclingEditingAgent`	see §6.6

NOTE

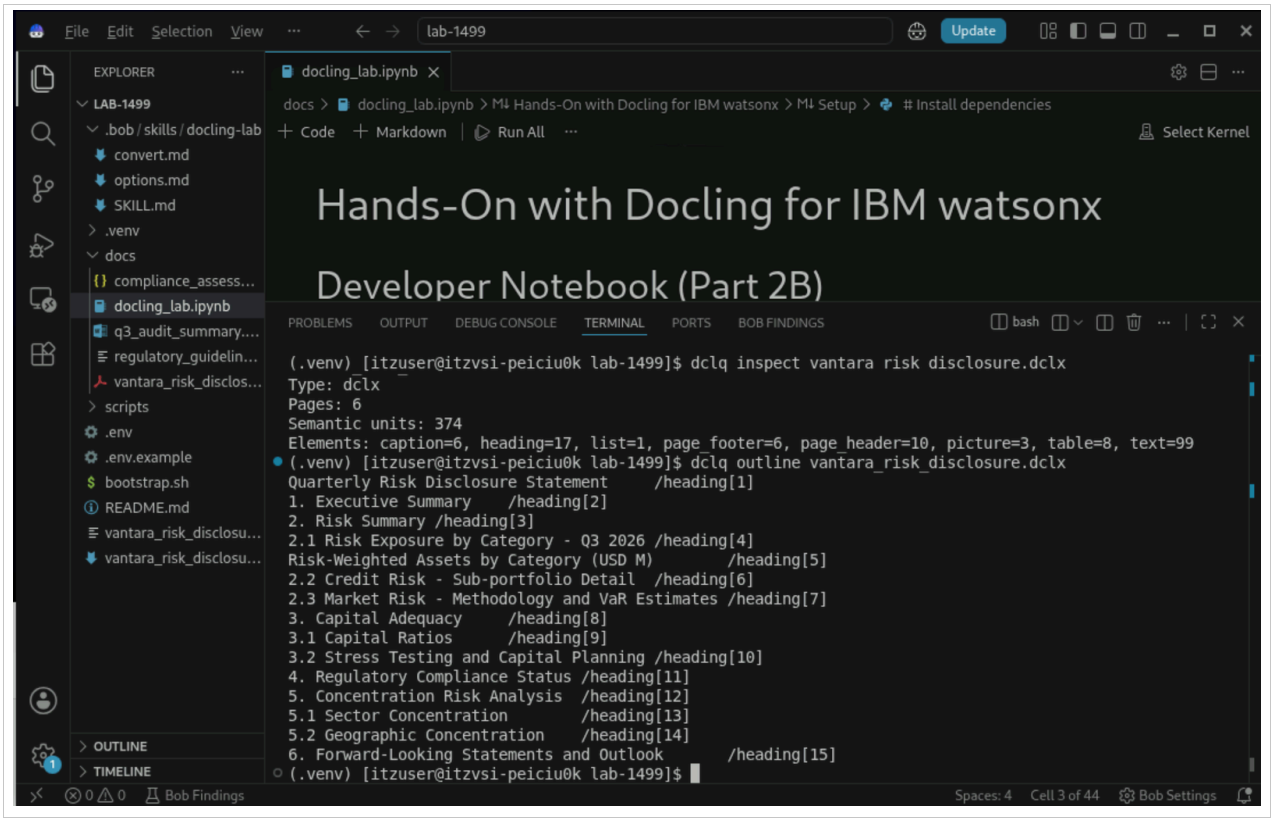
Notebook section 6 ends with a pointer worth remembering: every Docling chunk keeps the page and the bounding box it came from, which is what **visual grounding** is built on — highlighting in the original PDF exactly where an answer was found. Building that is out of scope today.

Notebook section 7 asks the language model for the form directly. Docling for IBM watsonx will soon do schema-based extraction as a **native** function of the service: you will send the schema with the documents and get the typed fields back.

6.4 Query DocLang from the command line

Notebook section 4 uses `dclq`, which is worth a minute on its own: it makes a converted document a thing you can grep. The notebook wrote one `.dclx` per document into `output/`, and the commands work in Bob's terminal too:

```
dclq inspect  output/vantara_risk_disclosure.dclx       # what is in it
dclq outline  output/vantara_risk_disclosure.dclx       # heading tree, with XPaths
dclq grep -i  'risk exposure' output/vantara_risk_disclosure.dclx
dclq outline  output/q3_audit_summary.dclx              # a DOCX, same kind of tree
```

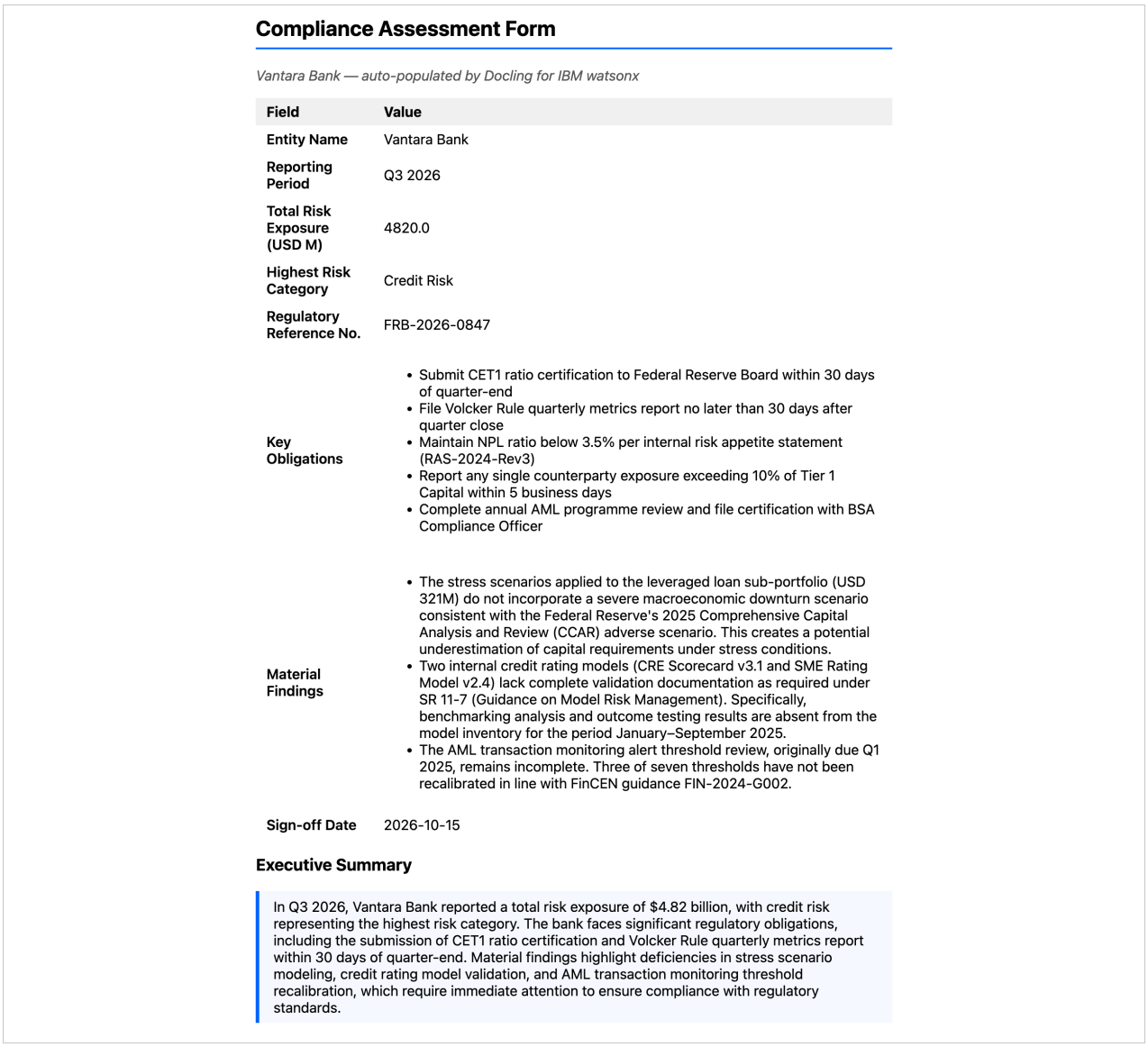


Screen 27. `inspect` then `outline`. The first counts what the document is made of — semantic units, headings, tables, and the page headers and footers counted separately as furniture. The second prints the heading tree with the XPath that addresses each one.

> **TIP**
>
> `dclq grep` searches *semantic units*, not lines. A match inside a table cell tells you the cell, not the character offset — which is the difference between finding a number and being able to say what it is a number of.

6.5 The filled form

Notebook sections 7 and 8 are where the three documents come together. The extraction returns one `ComplianceForm`; the summary is written from it, and both go into an HTML report the notebook displays and saves as `output/compliance_assessment_form_filled.html`.



Compliance Assessment Form

Vantara Bank — auto-populated by Docling for IBM watsonx

Field	Value
Entity Name	Vantara Bank
Reporting Period	Q3 2026
Total Risk Exposure (USD M)	4820.0
Highest Risk Category	Credit Risk
Regulatory Reference No.	FRB-2026-0847
Key Obligations	• Submit CET1 ratio certification to Federal Reserve Board within 30 days of quarter-end • File Volcker Rule quarterly metrics report no later than 30 days after quarter close • Maintain NPL ratio below 3.5% per internal risk appetite statement (RAS-2024-Rev3) • Report any single counterparty exposure exceeding 10% of Tier 1 Capital within 5 business days • Complete annual AML programme review and file certification with BSA Compliance Officer
Material Findings	• The stress scenarios applied to the leveraged loan sub-portfolio (USD 321M) do not incorporate a severe macroeconomic downturn scenario consistent with the Federal Reserve's 2025 Comprehensive Capital Analysis and Review (CCAR) adverse scenario. This creates a potential underestimation of capital requirements under stress conditions. • Two internal credit rating models (CRE Scorecard v3.1 and SME Rating Model v2.4) lack complete validation documentation as required under SR 11-7 (Guidance on Model Risk Management). Specifically, benchmarking analysis and outcome testing results are absent from the model inventory for the period January–September 2025. • The AML transaction monitoring alert threshold review, originally due Q1 2025, remains incomplete. Three of seven thresholds have not been recalibrated in line with FinCEN guidance FIN-2024-G002.
Sign-off Date	2026-10-15

Executive Summary

> In Q3 2026, Vantara Bank reported a total risk exposure of $4.82 billion, with credit risk representing the highest risk category. The bank faces significant regulatory obligations, including the submission of CET1 ratio certification and Volcker Rule quarterly metrics report within 30 days of quarter-end. Material findings highlight deficiencies in stress scenario modeling, credit rating model validation, and AML transaction monitoring threshold recalibration, which require immediate attention to ensure compliance with regulatory standards.

Screen 28. The report from notebook section 8: the eight fields, with values from all three documents, and the executive summary written from them.

> **TIP**
>
> Look at the two fields that appear in two documents. The model was given all three documents at once and returned one value for each — it made a choice. In a real compliance pipeline that choice is the thing an auditor would ask about.

6.6 If time permits — fill the form with Docling Agent

Notebook section 9 builds the empty form as a `DoclingDocument` and lets `DoclingEditingAgent` fill its table from the extracted values. The agent runs on watsonx.ai through a small backend defined in the notebook. It saves two files in `output/`:

- `compliance_assessment_form_empty.dclx` — the form before the agent touched it
- `compliance_assessment_form_agent_filled.dclx` — the same form, filled

Drag both onto doclang.ai/viewer and compare them: the structure is the same, and the agent wrote the values into the cells of the table.

> **YOU NOW HAVE**
>
> Three documents converted and queried as DocLang, a RAG pipeline that cites its sources, and the compliance assessment form filled by code you can read — with every model on watsonx.ai. Go to **Section 7**.

7 Compare the two routes

Five minutes, everyone. Both tracks filled the same eight fields from the same three documents. Put the two routes side by side — compare with your neighbour if they took the other track.

	Track A — IBM Bob	Track B — Python notebook
Who reads the documents	a language model, through the structure the skill returns	a language model, through the Markdown of the three `DoclingDocument`s
Where the form is defined	in your prompt, as a list of field names	in code, as a Pydantic model with types and descriptions
What you can check	the trace: which skill, which command, which file	every cell: the input, the prompt, the schema, the output
Running it again	ask again — the answer may be worded differently	run the cells again — same code, same schema
What it is good for	exploring a bundle you have not seen, quickly	a pipeline you will run every quarter

Two questions to answer before you leave:

- **Did the two fields that appear twice agree?** `total_risk_exposure_usd` and `regulator_reference_number` are each in two documents. Did your route tell you that, or did it pick one silently?
- **Where did the structure matter?** Both routes read a table cell as a table cell. Find one field that would have been a guess if the table had collapsed into text.

7.1 What you just did

In seventy-five minutes, without a GPU and without writing a parser:

- Converted three documents in three formats into one structure.
- Exported them to an ISO standard and inspected them down to individual table cells.
- Pulled eight fields out of three documents into one form — by asking, or in code.
- Generated a narrative summary and an audit-ready form.

The part worth carrying away is the ordering. Every one of those steps was possible because the first one produced structure rather than text — and the reason the last step can be trusted at all is that the field it printed came out of a table cell that was recognised as a table cell.

YOU NOW HAVE

A filled compliance assessment form, a written summary, and a view of two ways to get there — from three documents that had nothing in common but their subject.

8 Where to take it next

What you built today is the front half of most document systems. Three directions, in rough order of how often they come up:

- **Retrieval over a real corpus.** Section 6 of the notebook is a RAG pipeline over three documents; the same code over three thousand is a different engineering problem but not a different design. Chunk quality is what decides whether it works, and the provenance every chunk carries is what lets you add visual grounding.
- **Extraction against a schema you own.** The `ComplianceForm` model is eight fields of a fictional form. Point it at a form your own organisation actually fills in by hand, and the interesting question becomes which fields are reliably extractable and which are not.
- **Documents in, documents out.** Both tracks hint at it: Docling can produce a `DoclingDocument`, not just consume one, and an agent can edit one. A pipeline that reads a bundle and emits a filled form is the same tooling in both directions.

And if you want the event-driven version of this — a bucket upload that converts, chunks, enriches and indexes with nobody running anything — that is **LAB-2775**, in the same repository.

9 Did not finish the lab?

Nothing here disappears when the session ends.

- The lab material is public: github.com/IBM/txc2026-docling, in `lab-1499/`. Everything in this guide is there, including the notebook.
- **Your Docling for IBM watsonx trial is yours** and keeps working after the lab. It is on your own IBMid, not on anything the conference lent you.
- The self-paced lab room has the same VM image. The bootstrap in §2.2 is one command and puts you back where you left off.

If you would rather continue on your own machine, the same lab runs locally — install `docling` instead of the client and point the notebook at a local converter. The models are large; everything else is identical.

10 Getting help

10.1 Ask the lab instructor

Raise a hand. The instructors can see the state of the shared environments, and they carry spare Docling credentials for anyone whose trial is slow to provision — which is the one problem in this lab you cannot fix yourself.

10.2 Common problems

Symptom	Likely cause	What to do
A command is *not found* right after the bootstrap	this terminal started before the tooling was installed	`export PATH="$HOME/.local/bin:$PATH"`, or open a new terminal
The trial never becomes active	provisioning is slow, or the IBMid needs confirming	check for a confirmation e-mail, then ask the instructor for shared credentials and carry on
`X Missing credentials` from `test_connection.py`	`.env` does not exist, or a variable name is misspelt	`cp .env.example .env` and fill in both values — §3.6
The notebook setup cell says *Set these variables* for the `WX_` ones	the shared watsonx.ai file is not on your VM	check that `~/lab-1499-watsonx.txt` exists; if it does not, raise a hand — the instructors put it there
`401` or `403` from the service	the API key is wrong, or a newer one was generated after you copied it	generate a fresh key and update `.env` — §3.5
A connection error, but the key looks right	the dashboard URL was copied instead of the service URL	take the URL from **API examples**, including its path — §3.5
Bob does not seem to know about Docling	the skill was not picked up, or its tools were never approved	`ls .bob/skills/docling-lab/SKILL.md`, reload the workspace, and approve the tools when asked — §5.3
Bob answers from the file name rather than the content	the conversion did not actually run	ask it to convert the document and return the key first, then ask your question
Bob will not answer at all	it is installed but not signed in	**Log in to Bob** in the right-hand panel — §5.2
The notebook cannot import `docling`	the notebook is running on a different kernel than the environment, or the setup cell was skipped	**Select Kernel** → the lab's `.venv`, then run the **Setup** cells — §6.1, §6.2
A `.dclx` file will not open in the viewer	the download is the archive, not the document inside it	unpack it and drag the `.dclx` itself onto the viewer
The extracted numbers are wrong	this is a model reading a structure, not a parser	check them against the document — noticing this is part of the exercise, not a failure of it

10.3 Useful commands

```
cd ~/txc2026-docling/lab-1499
source .venv/bin/activate

python scripts/test_connection.py           # is the service reachable, with my
key?

dclq inspect  doc.dclx                      # what is in this document
dclq outline  doc.dclx                      # heading tree, with XPaths
dclq grep -i  'PATTERN' doc.dclx            # search semantic units
dclq show     doc.dclx '/heading[2]' --section

curl -L ibm.biz/txc26-1499-bootstrap | bash -   # safe to re-run; keeps your .env
```

11 Reference material

	Where
This lab's repository	github.com/IBM/txc2026-docling → `lab-1499/`
Docling for IBM watsonx — the product	ibm.com/products/docling
Docling for IBM watsonx — free trial	ibm.biz/docling-for-watsonx-trial-txc26
Docling for IBM watsonx — the documentation	developer.dcls.saas.ibm.com
Docling (open source)	docling.ai
Agent skills documentation	docling-project.github.io/docling/usage/agent_skills
DocLang Viewer	doclang.ai/viewer
`dclq` command line	github.com/docling-project/docling-core
Docling technical report	arxiv.org/abs/2501.17887
The event-driven companion lab	**LAB-2775** — From Bucket to RAG

Tell us how it went. Scan the QR code on the classroom screen, or use the feedback link your instructor gives you. If you build something with this we would like to hear about it — the authors' addresses are on the cover.